

Reviser: Revision-Capable Text Generation via Autoregressive Cursor Actions

Sean Diab
Independent Researcher
diabsean2005@gmail.com

April 27, 2026

Abstract

Revision-capable generation is appealing because it can insert or revise earlier content, but many non-autoregressive and edit-based approaches obtain this flexibility through repeated sequence-level computation. We propose Reviser, a decoder-only Transformer that generates a response as a sequence of cursor-relative actions on a mutable canvas. At each step, Reviser predicts exactly one action token: INSERT(token), MOVE(Δ), or STOP, and is autoregressive over edit-history actions rather than final text order.

This design enables genuinely non-monotonic generation while preserving a simple next-action interface. On a continuation benchmark, Reviser is strongly preferred to SEDD and MDLM in our arena evaluations, and trajectory statistics confirm that the model performs frequent backward moves and mid-canvas insertions rather than merely emulating end-append decoding. Against size-matched autoregressive baselines, Reviser is competitive at both the 100M and 300M scales. Under our shared FLOPs convention, Reviser also requires substantially less inference compute than representative multi-pass refinement and diffusion-style baselines.

1 Introduction

Autoregressive Transformers dominate text generation, but they are structurally biased toward producing content in final left-to-right order: once a clause is emitted, correcting it typically requires generating additional text after it rather than directly revising earlier content (Vaswani et al., 2017; Brown et al., 2020). Revision-capable and non-autoregressive methods promise post-hoc correction and increased parallelism, yet in practice they frequently incur large overhead in the forms of multiple full-sequence refinement passes, scoring every insertion slot, or many-step sampling procedures (Ghazvininejad et al., 2019; Gu et al., 2019; Li et al., 2022; Lou et al., 2024; Sahoo et al., 2024). This paper asks: can we get revision-style generation while keeping total compute close to a standard AR Transformer?

We propose Reviser, which generates by executing a stream of simple cursor edits on a mutable canvas. In the primary implementation, Reviser uses only insert and move actions (plus STOP), where “revision” means non-left-to-right insertion into earlier positions; although Reviser naturally supports richer edit operators (e.g., DELETE/REPLACE/span edits), we focus here on this minimal insert+move (+STOP) instantiation and leave destructive editing to future work. The model is a standard decoder-only transformer, but crucially, it is autoregressive over action tokens (the edit history), not over the final text token order. Because the cursor can move and insert earlier content,

Reviser is not constrained to left-to-right generation in the final text order, enabling revision-like behavior while keeping each model call “normal-sized.”

Rather than outputting a distribution over all positions (or all insertion slots) at each step, Reviser outputs exactly one cursor-relative action token per step. Importantly, the transformer trunk attends only to the action-history sequence H_t ; it never attends to canvas tokens directly. The canvas is implicit in the history of executed actions and is accessed only through validity masking. This keeps each model call comparable to a standard AR step (transformer trunk + one head). The remaining overhead is the number of action steps, which can be close to the output length in an insert-dominant regime.

Beyond text continuation, the cursor-action formulation also suggests applications to structured editing tasks such as code or document modification. Modern agent systems often edit files by repeatedly proposing diffs or patches over entire sequences (Yang et al., 2024; Gauthier, 2024), which can require multiple full-sequence passes. In contrast, Reviser operates through localized insert and move actions on a mutable canvas, which may offer a more natural interface for incremental editing. We do not evaluate this setting in the current work, but view it as a promising direction for future research.

A concrete example helps illustrate the behavior. In one 300M Reviser generated trajectory (300M Reviser Example 1 in Appendix E):

Before edit

Information is deemed correct and is subject to change. Real estate listings obtained from third party sources are for | consumers' personal purchasing decisions and should...

Actions (steps 204-209)

MOVE(-16) + **INSERT**(at the time of publishing)

After move

Information is deemed correct at the time of publishing | and is subject to change.
Real estate listings obtained from third party sources are for...

This edit makes the statement more precise. Appendix E provides full qualitative examples and trajectories.

Contributions.

1. We introduce Reviser, a cursor-action generator that is autoregressive over edit-history actions rather than final text order.
2. We formalize the canvas state, cursor-based edit operators, validity masking, and deterministic executor updates for the insert+move+STOP setting.
3. We describe an obfuscation–restoration supervision scheme for training next-action predictors on edit trajectories.
4. We show empirically that Reviser produces genuinely non-monotonic trajectories, with frequent backward moves and mid-canvas insertions.
5. We evaluate Reviser on continuation against diffusion and autoregressive baselines, and analyze quality, length behavior, and trajectory statistics.
6. We provide an analytic FLOPs-based comparison showing that Reviser is substantially cheaper than representative multi-pass refinement and diffusion-style baselines under a shared convention.

Section 2 states the problem setup and design goals, and Section 3 motivates the compute tradeoffs behind our design. Section 4 situates Reviser in prior work. Section 5–Section 6 formalize Reviser and the decoding procedure, and Section 7 describes supervision via obfuscation–restoration trajectories. Section 8 reports empirical results, while Section 9–Section 11 discuss limitations, future directions, and conclusions. The Appendix provides additional variants, full FLOPs/accounting derivations, qualitative examples, and reproducibility details.

2 Problem Setup and Design Goals

Reviser supports prompt-conditioned generation in general; in this work, we instantiate it as a prefix-seeding text continuation model: given an input prefix $x = (x_1, \dots, x_m)$, the goal is to generate the continuation sequence.

To make the prefix visible to the history-only model, we write it into both the canvas and the action history via a deterministic insert-only prefix-seeding action sequence:

$$\mathcal{A}^{\text{pref}}(x) = (\text{INSERT}(x_1), \text{INSERT}(x_2), \dots, \text{INSERT}(x_m)).$$

Executing $\mathcal{A}^{\text{pref}}(x)$ from a blank canvas places each prefix token on the canvas in order, sets the cursor at position m , and makes the full prefix content visible to the transformer through its action-history input H_t . Continuation decoding then begins from this seeded state, with a validity mask that prevents edits to the prefix region.

Our design goal is to support revision-capable generation, the ability to insert or modify earlier content during decoding, while keeping the compute profile close to a standard AR transformer.

We aim to avoid common sources of overhead in prior NAR approaches: (i) repeated full-sequence refinement passes, (ii) per-step scoring over all positions/slots, (iii) large per-position action heads, and (iv) sampling procedures requiring many denoiser steps. Reviser instead makes a single cursor-relative decision per step using a standard transformer trunk and a single action head.

We evaluate (a) generation quality (evalPPL under GPT-2 Large and arena pairwise win rates), (b) compute (FLOPs), and (c) trajectory behavior (steps/token, move fraction, cursor travel, insertion locations).

3 Compute Tradeoffs in Existing NAR and Edit-Based Transformers

Many non-autoregressive and edit-based Transformers obtain flexibility or parallelism by repeating expensive sequence-level computation multiple times per response (Lee et al., 2018; Ghazvininejad et al., 2019; Gu et al., 2019; Li et al., 2022; Lou et al., 2024; Sahoo et al., 2024). Under a shared FLOPs convention, this often leads to substantially higher total inference compute than a standard autoregressive baseline. Our goal in this section is not to claim that all such methods are inefficient in practice, but rather to highlight a common tradeoff: repeated full-sequence prediction can raise total compute even when sequential dependence is reduced.

In our FLOPs accounting (Appendix B), an AR baseline produces a length- n response with one full-sequence evaluation. By contrast, most NAR families run multiple full-sequence evaluations (refinement iterations, insertion rounds, denoising steps, etc.), so they repeatedly pay the dominant Transformer trunk cost over all n positions.

Symbol	Meaning
$x = (x_1, \dots, x_m)$	input prefix (conditioning context)
m	prefix length in tokens
$\mathbf{x} = (a_1, \dots, a_T)$	restoration trajectory (total action sequence)
$B = (b_1, \dots, b_{n_{\text{obf}}})$	obfuscation trajectory (action sequence)
V_c	insert token vocabulary
V_a	action vocabulary ($V_c \cup \mathcal{M} \cup \{\text{STOP}\}$)
$C_t = (c_{t,1}, \dots, c_{t,\ell_t})$	canvas token sequence at step t
ℓ_t	canvas length at step t
$u_t \in \{0, \dots, \ell_t\}$	cursor boundary index (between tokens)
H_t	edit-history action sequence at continuation step t (includes prefix-seeding actions)
n	final token output length (reference; not edit-history length)
p_{move}	fraction of generated actions that are MOVE actions
n_{eff}	effective action length, $n_{\text{eff}} \triangleq \frac{n}{1-p_{\text{move}}}$
T_{rest}	restoration trajectory length in actions (including STOP)
T_{dec}	diffusion-style reverse/denoising step count
T_{mp}	Mask-Predict/CMLM refinement-iteration count
$F_{\text{mult}}(a, b, c)$	MAC-count proxy for matmul $[a \times b] \cdot [b \times c]$
$F_{\text{transformer}}^{\text{full}}(n)$	trunk cost for one full-attention Transformer pass at length n
$F_{\text{transformer}}^{\text{causal}}(n)$	trunk cost for one causal-attention Transformer pass at length n
$F_{\text{infer}}^{\mathcal{M}}(n)$	total inference compute for method $\mathcal{M}$
$I_{\text{infer}}^{\mathcal{M}}$	inference multiplier vs AR: $F_{\text{infer}}^{\mathcal{M}}/F_{\text{infer}}^{\text{AR}}$

Table 1: Notation used throughout the paper (core variables).

A single full-sequence evaluation computes hidden states for all n positions and then applies a vocab projection, yielding $n \times |V|$ logits. At modern scales, both terms matter: the full Transformer pass is expensive, and the vocab head is also large and non-negligible (e.g., $|V| \approx 50\text{k}$). Methods that perform R refinement/denoising iterations therefore multiply both costs by roughly R per response, often with additional task-specific heads, slot-scoring loops, or auxiliary passes. In several families, R is large: for example, CMLM commonly uses around $R = 10$ refinement passes, while diffusion-style decoders often use $R = T_{\text{dec}} \in [200, 2000]$ reverse steps. This repeated sequence-level evaluation is highly inefficient at inference time, especially for long outputs or throughput-limited deployment.

For overall efficiency, especially at scale, the dominant quantity is often total FLOPs per response. If a method is $10\times$ more compute-intensive, modest systems-level scheduling gains usually do not offset that gap. Total compute is therefore the primary constraint for training cost, energy, and throughput-limited deployments. We report a compute multiplier

$$I_{\text{infer}}^{\mathcal{M}} \triangleq \frac{F_{\text{infer}}^{\mathcal{M}}}{F_{\text{infer}}^{\text{AR}}},$$

where $F_{\text{infer}}^{\mathcal{M}}$ is the total FLOPs to generate a full length- n response under method $\mathcal{M}$'s decoding procedure. In this section and appendix, we focus on $F_{\text{infer}}^{\mathcal{M}}$ and $I_{\text{infer}}^{\mathcal{M}}$ as the primary compute indicators.

Table 2 summarizes our computed multipliers relative to the AR baseline (derivations in the Appendix). With few exceptions, popular NAR Transformers require significantly more total compute than AR, often by an order of magnitude or more. The methods that appear closer to AR in compute typically rely on highly parallel, one-/few-shot predictions; in practice these variants struggle

Method (family / variant)	Decoding structure	$I_{\text{infer}}^{\mathcal{M}}$	Parallel decoding issue?
AR (baseline)	n AR steps	1.00	No
Reviser (this work)	T_{rest} AR-style steps	1.25–1.50 [†]	No
LevT (iterative edit/refine)	$R \in \{5, 10\}$ passes	6.91–19.40	Yes
InsT (balanced-tree)	$\log_2 n$ passes	2.02	Yes
InsT (serial)	n passes	65.01	No
CMLM / Mask-Predict ($T_{\text{mp}}=10$)	10 passes	11.86	Yes
Diffusion-LM ($T_{\text{dec}}=200\text{--}2000$)	200–2000 passes	140.51–1,402.36	No
One-shot NAT (few-pass)	1 enc + 1 dec pass	1.96	Yes

Table 2: Analytic inference-compute summary under a shared FLOPs convention (computed in the Appendix). $I_{\text{infer}}^{\mathcal{M}} \triangleq F_{\text{infer}}^{\mathcal{M}}/F_{\text{infer}}^{\text{AR}}$ is the total inference FLOPs multiplier of method $\mathcal{M}$ relative to AR; values are structural estimates, not measured wall-clock latency. [†] Range shown for $p_{\text{move}} \in \{0.20, 0.28, 0.33\}$ at $n = 128$ (including observed move fractions from Table 10); see Appendix C.1.2. For AR/Reviser, “steps” denotes KV-cached next-token/next-action decoding; “passes” for other methods denotes repeated full-sequence evaluations. “Parallel decoding issue” marks families whose highly parallel variants often underperform due to global coordination constraints, commonly motivating additional refinement passes. Further details of all calculations are provided in Appendix B.

due to the parallel decoding issue (below), and are commonly augmented with iterative refinement, which reintroduces the repeated-pass overhead.

The most compute-efficient NAR variants attempt to predict many (or all) output tokens simultaneously. This creates a global coordination problem: each position must choose a token that is compatible with the (unknown) choices at other positions. Formally, one-shot NAR variants often behave like a product-of-marginals approximation (Gu et al., 2018),

$$p(y \mid x) \approx \prod_{i=1}^n p(y_i \mid x),$$

which cannot reliably enforce inter-token constraints without extra structure (latents, constraints, or iterative refinement). In practice, this manifests as agreement errors, repetition, missing required entities, and internal contradictions, motivating additional passes that raise compute.

Reviser avoids the parallel-decoding coordination issue by predicting one cursor-relative action token at a time, conditioned on the full prior edit history. This keeps global coherence through autoregressive dependence while still allowing non-monotonic canvas edits. It also avoids repeated full-sequence refinement: each step is a standard AR-style next-action pass, rather than repeated re-evaluation over all n output positions. If the move fraction is p_{move} , producing n final tokens takes an expected $n_{\text{eff}} = \frac{n}{1-p_{\text{move}}}$ action steps, so

$$I_{\text{infer}}^{\text{Reviser}}(n) \approx \frac{F_{\text{infer}}^{\text{AR}}(n_{\text{eff}})}{F_{\text{infer}}^{\text{AR}}(n)}.$$

For $p_{\text{move}} \in \{0.20, 0.28, 0.33\}$ and $n = 128$, this gives $n_{\text{eff}} \in \{160, 177.78, 191.04\}$ and $I_{\text{infer}}^{\text{Reviser}}(128) \in \{1.25, 1.39, 1.50\}$.

4 Related Work

Reviser is closest to non-autoregressive and partially autoregressive generation that relaxes strict left-to-right decoding. Early one-shot NAT models improve parallelism but face multimodality and coordination issues (Gu et al., 2018); iterative variants such as deterministic refinement and Mask-Predict improve quality by repeating full-sequence passes (Lee et al., 2018; Ghazvininejad et al., 2019). Reviser targets the same non-monotonic capability, but keeps a one-action interface per step instead of repeated sequence-level prediction.

Our approach is also related to insertion and edit-based generation. Insertion Transformer and Levenshtein Transformer show that insertion/deletion operations can realize flexible generation orders (Stern et al., 2019; Gu et al., 2019). Text-edit tagging systems such as LaserTagger and FELIX frame generation as edit prediction over an existing sequence (Malmi et al., 2019; Mallinson et al., 2020). Many of these methods were introduced and evaluated primarily in sequence transduction settings (especially machine translation and grammatical-error-correction style tasks), rather than open-ended continuation generation. PIE-style post-editing formulations make a similar design choice by treating generation as targeted rewriting rather than pure left-to-right continuation. Reviser is in the same family of ideas, but uses explicit cursor actions with deterministic executor semantics.

Diffusion-style LMs provide another route to non-monotonic generation. Diffusion-LM, SEDD, and MDLM decouple final token order from AR factorization through denoising trajectories (Li et al., 2022; Lou et al., 2024; Sahoo et al., 2024). More recent semi-AR or block-diffusion variants, including LLaDA, Dream, and SDLN (analyzed in our Appendix FLOPs section), further explore the quality and throughput tradeoff by mixing iterative denoising with partial autoregressive structure. Latent-variable NAR models are also relevant. Flow-based sequence generators such as FlowSeq model conditional generation through invertible latent transformations instead of strict tokenwise AR decoding (Ma et al., 2019). This line is conceptually close in its goal of relaxing left-to-right constraints, though its modeling interface differs from explicit executable edit actions.

Reviser also relates to work on non-left-to-right autoregression. XLNet demonstrates permutation-based autoregressive objectives over factorization orders (Yang et al., 2019), and insertion-based AR models similarly depart from fixed final-token order (Stern et al., 2019). Reviser differs by making edit history itself the autoregressive object: one cursor-relative action per step, executed immediately on a mutable canvas.

Our positioning is therefore narrow: Reviser uses cursor-relative actions over a mutable canvas, predicts exactly one next action token at a time, and applies edits through a deterministic executor. This preserves revision capability while keeping the decoding interface close to standard next-token prediction and, empirically, much closer to AR compute than multi-pass refinement families.

5 Reviser Formalism: State, Actions, and Executor

Reviser maintains a mutable canvas and a cursor indicating the insertion boundary. At each step, the model reads the history of past actions (action tokens), applies validity masking using (C_t, u_t) , and predicts one next action token (Insert / Move / Stop). An external executor applies the action to update the canvas and cursor. In the prefix-seeding setting, the canvas and action history are pre-seeded with $\mathcal{A}^{\text{pref}}(x)$ before continuation decoding begins. Figure 1 illustrates one decoding step.

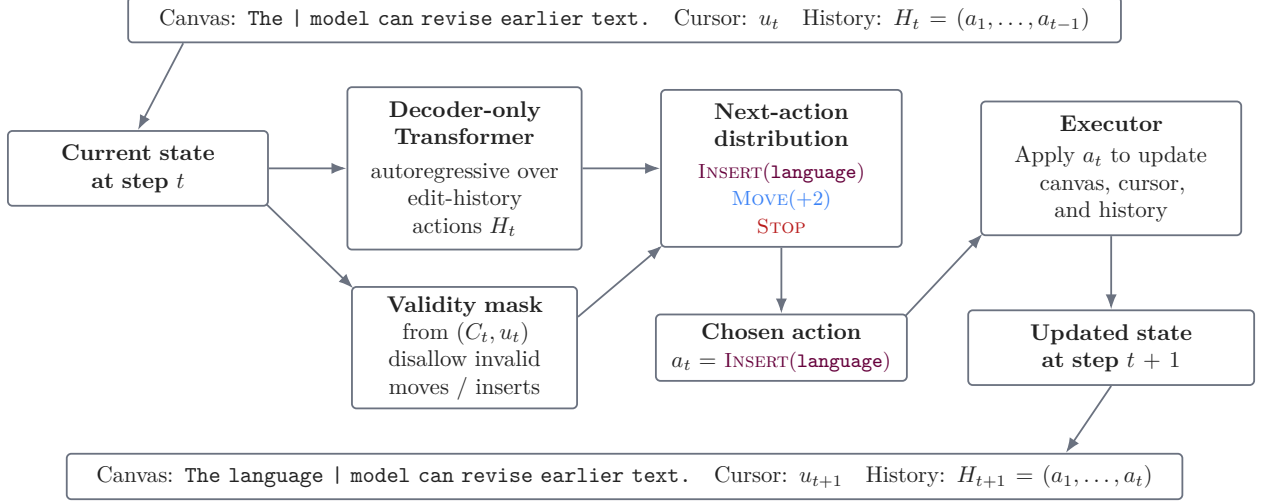


Figure 1: Reviser decoding at a single step. The model is autoregressive over edit-history actions rather than final text order. At each step it predicts exactly one cursor-relative action, applies a validity mask derived from the current canvas state, and then uses a deterministic executor to update the canvas, cursor, and history.

5.1 State and Canvas Representation

A generation state at step t is

$$s_t = (C_t, u_t, H_t),$$

where:

- $C_t = (c_{t,1}, \dots, c_{t,\ell_t})$ is the canvas token sequence at step t , with canvas length ℓ_t ,
- $u_t \in \{0, 1, \dots, \ell_t\}$ is the cursor index between tokens (0 means before the first token),
- H_t is the edit history at continuation step t (the action sequence executed so far). In the prefix-seeding setting, $H_1 = \mathbf{x}^{\text{pref}}(x)$ and, for $t \geq 1$, $H_t = (\mathbf{x}^{\text{pref}}(x), a_1, \dots, a_{t-1})$, where a_i are continuation actions.

Reviser uses a standard decoder-only transformer that consumes the sequence of past action tokens H_t and predicts the next action token a_t .

5.2 Action Space (Primary Implementation)

The action vocabulary is

$$V_a = \underbrace{V_c}_{\text{INSERT(token)}} \cup \underbrace{\mathcal{M}}_{\text{MOVE}(\Delta)} \cup \{\text{STOP}\}.$$

Here V_c is the normal token vocabulary (each token corresponds to an INSERT action), and $\mathcal{M}$ is a small discrete set of move actions (e.g., $\Delta \in \{\pm 1, \pm 2, \pm 4, \dots\}$). STOP ends generation.

5.3 Executor: Canvas Update Mathematics

Let $C_t = (c_{t,1}, \dots, c_{t,\ell_t})$ and cursor $u_t \in \{0, \dots, \ell_t\}$.

For an insertion of token $x \in V_c$ at cursor boundary $u_t \in \{0, \dots, \ell_t\}$,

$$\text{INSERT}(x) : C_{t+1} = (c_{t,1}, \dots, c_{t,u_t}, x, c_{t,u_t+1}, \dots, c_{t,\ell_t}), \quad u_{t+1} = u_t + 1.$$

For a displacement $\Delta \in \mathbb{Z}$ chosen from $\mathcal{M}$,

$$\text{MOVE}(\Delta) : C_{t+1} = C_t, \quad u_{t+1} = u_t + \Delta.$$

We enforce the post-state constraint $u_{t+1} \in \{0, \dots, \ell_{t+1}\}$ via validity masking (for MOVE, $\ell_{t+1} = \ell_t$).

STOP : terminate and output C_t .

5.4 Validity Masking

At each step, Reviser constructs a valid-action mask based on (C_t, u_t) , for example:

- forbid moves $\text{MOVE}(\Delta)$ such that $u_t + \Delta \notin \{0, \dots, \ell_{t+1}\}$ (equivalently $\{0, \dots, \ell_t\}$ for MOVE),
- forbid inserting if ℓ_t has reached a maximum length,
- optionally forbid inserting certain reserved tokens.

Masking is applied to the action logits before sampling/argmax.

6 Model

6.1 Edit-History Transformer

Let H_t be the edit-history action tokens at continuation step t . In prefix-seeding decoding, $H_t = (\mathfrak{A}^{\text{pref}}(x), a_1, \dots, a_{t-1})$. We use a single shared embedding table $E \in \mathbb{R}^{|V_a| \times d}$ for all action tokens (including token-valued INSERT actions, MOVE actions, and STOP), and a positional embedding table $P^{(H)} \in \mathbb{R}^{T_{\max} \times d}$ for edit-history positions. The history-token embeddings are

$$e_\tau = E[a_\tau] + P^{(H)}[\tau], \quad \tau = 1, \dots, t-1.$$

A standard causal transformer processes $(e_1, \dots, e_{t-1})$ and produces hidden states $(h_1, \dots, h_{t-1})$. The next-action logits are

$$r_t = W_{\text{out}} h_{t-1} + b_{\text{out}}, \quad p(a_t | H_t, C_t) = \text{softmax}(r_t + m_t),$$

where m_t applies the validity mask computed from (C_t, u_t) (invalid actions get $-\infty$). In our experiments, the trunk depends only on H_t ; the canvas enters through the executed action sequence (which determines what the model has seen) and through masking. Optional designs that add an explicit canvas summary μ_t (attention pooling) or cross-attention to canvas token representations are described in Appendices A.2 and A.3.

In the prefix-seeding setting, the prefix $x = (x_1, \dots, x_m)$ enters the model exclusively through the seeded action history: the m prefix-seeding actions $\mathfrak{A}^{\text{pref}}(x) = (\text{INSERT}(x_1), \dots, \text{INSERT}(x_m))$ occupy the first m positions of H_t , so the transformer’s self-attention over edit-history tokens directly attends to the full prefix content. No separate encoder or prefix embedding is required; the prefix is fully visible through the standard action-history input stream, and a validity mask applied during continuation decoding prevents the model from editing the prefix region of the canvas.

Algorithm 1 Reviser decoding (insert+move primary implementation)

Require: Input prefix $x = (x_1, \dots, x_m)$ (may be empty), max steps $T_{\max}$, max canvas length $L_{\max}$

```
1: Phase 1: Prefix seeding
2: Initialize canvas  $C \leftarrow ()$ , cursor  $u \leftarrow 0$ , history  $H \leftarrow ()$ 
3: for  $i = 1, \dots, m$  do
4:   Apply INSERT( $x_i$ ):  $C \leftarrow (C, x_i)$ ,  $u \leftarrow u + 1$ ,  $H \leftarrow (H, \text{INSERT}(x_i))$ 
5: end for
6: Set  $C_1 \leftarrow C$ ,  $u_1 \leftarrow m$ ,  $H_1 \leftarrow H$   $\triangleright H_1 = \mathfrak{A}^{\text{pref}}(x)$ ; canvas =  $x$ ; cursor at  $m$ 
7: Phase 2: Continuation decoding
8: for  $t = 1, 2, \dots, T_{\max}$  do
9:   Compute action logits  $r_t \leftarrow f_{\theta}(H_t)$   $\triangleright$  optionally supply a canvas summary; see Appendix
10:  Construct validity mask  $m_t \leftarrow \text{MASK}(C_t, u_t, L_{\max}, |x|)$   $\triangleright$  disallow edits to prompt-prefix
11:  Choose action  $a_t \sim \text{softmax}(r_t + m_t)$ 
12:  if  $a_t = \text{STOP}$  then
13:    return  $C_t$ 
14:  else
15:    Apply executor update  $(C_{t+1}, u_{t+1}) \leftarrow \text{EXEC}(C_t, u_t, a_t)$ 
16:    Append to history  $H_{t+1} \leftarrow (H_t, a_t)$ 
17:  end if
18: end for
19: return  $C_{T_{\max}+1}$   $\triangleright$  fallback if no STOP
```

6.2 Generation Algorithm

Algorithm 1 gives the decoding loop. The loop terminates when STOP is chosen (or a maximum step budget is reached). Algorithm 1 has two phases. Phase 1 (prefix-seeding) is deterministic: it replays the prefix as m insert actions, writing x onto the canvas and building $H_1 = \mathfrak{A}^{\text{pref}}(x)$ in the action history. No model call is made during Phase 1. Phase 2 (continuation decoding) is autoregressive: the model conditions on the full seeded history H_t (which begins with $\mathfrak{A}^{\text{pref}}(x)$) and samples continuation actions one at a time. Because the prefix tokens appear as action-history entries, the model’s self-attention directly observes prefix content at every continuation step without any additional encoder.

7 Learning and Training

7.1 Worked Example: Edit History Trajectory

This section first shows a sample non-monotonic trajectory in Figure 2, then gives a separate worked obfuscation–restoration trajectory in Table 3. We render the cursor as a vertical bar “|” between tokens.

Reviser is trained to predict the next action token under teacher forcing on action trajectories. Training follows the same two-phase structure as decoding: a deterministic prefix-seeding phase $\mathfrak{A}^{\text{pref}}(x)$ seeds the canvas and action history, followed by a learned restoration phase $\mathfrak{A}^{\text{rest}}(y)$ over the continuation target; NLL is computed only over $\mathfrak{A}^{\text{rest}}(y)$. Given a restoration trajectory $\mathfrak{A}^{\text{rest}} =$

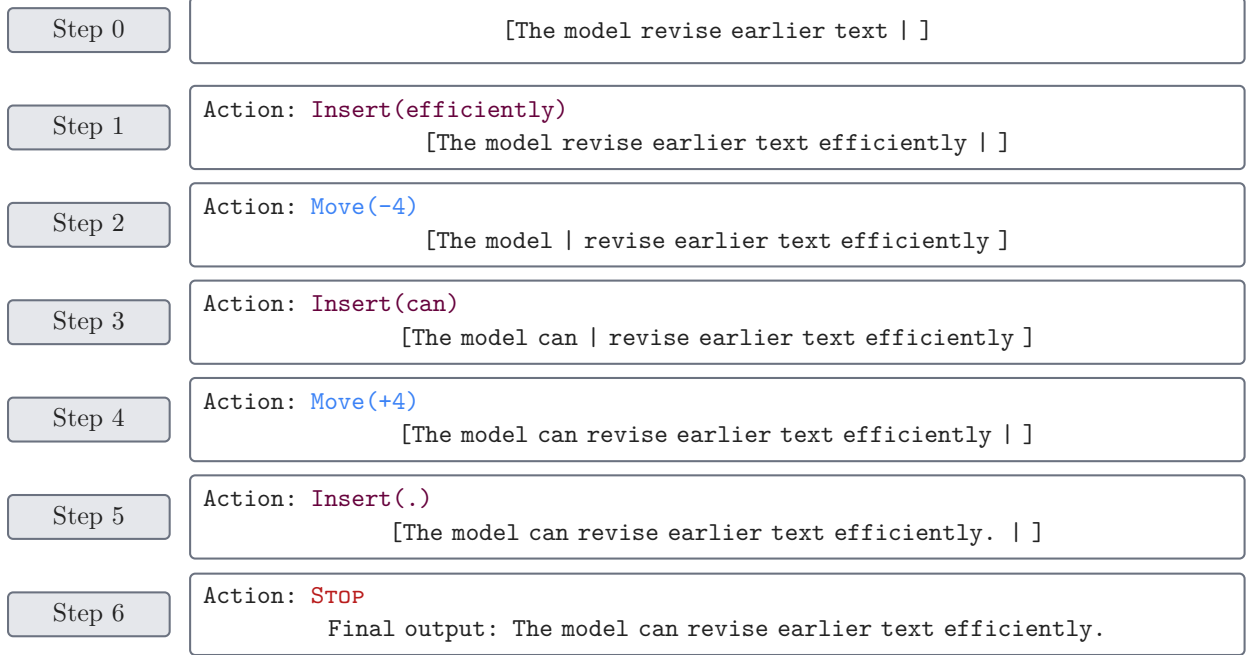


Figure 2: Illustrative Reviser trajectory. The model need not generate in final left-to-right order: it first appends a word to the end (Step 1), then moves the cursor backward (Step 2), inserts another word into the middle of the canvas (Step 3), returns to the end (Step 4), appends a period (Step 5), and stops (Step 6). This non-monotonic pattern, backward moves followed by mid-canvas insertions, is common in large-scale trajectory statistics, not an edge case.

$(a_1, \dots, a_T)$, we minimize the standard negative log-likelihood:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(a_t | s_t).$$

7.2 Obfuscation–Restoration Supervision (Main Training Procedure)

Algorithm 2 specifies the offline trajectory-construction procedure used for supervision. It samples an obfuscation sequence over the continuation canvas, records deleted tokens, and then builds the restoration sequence by reversing and inverting the obfuscation actions, finally appending STOP.

We construct training data using paired trajectories (see Table 3 for a concrete side-by-side example): (i) an obfuscation trajectory that transforms a target sequence into a blank canvas state, and (ii) a restoration trajectory $\star^{\text{rest}}(y)$ that transforms the blank state back to the target. The full supervised sequence for a document split into prefix x and continuation target y is

$$\star = (\star^{\text{pref}}(x), \star^{\text{rest}}(y)),$$

where $\star^{\text{pref}}(x) = (\text{INSERT}(x_1), \dots, \text{INSERT}(x_m))$ is the deterministic prefix-seeding phase and $\star^{\text{rest}}(y)$ is the restoration trajectory over the continuation target $y = (y_1, \dots, y_n)$. In richer action spaces that include temporary obfuscation-only insertions later removed by restoration deletes, those transient insertions are excluded from the loss because they are often random/noise tokens introduced only to create states that include delete actions, not meaningful target-response content.

Obfuscation (target $\rightarrow$ blank)	Restoration (blank $\rightarrow$ target)
1. Initial [Hi, how are you?]	1. Initial []
2. DELETE [Hi how are you?]	2. INSERT(how) [how]
3. DELETE [how are you?]	3. INSERT(are) [how are]
4. MOVE(+4) [how are you?]	4. INSERT(you) [how are you]
5. DELETE [how are you]	5. INSERT(?) [how are you?]
6. DELETE [how are]	6. MOVE(-4) [how are you?]
7. DELETE [how]	7. INSERT(Hi) [Hi how are you?]
8. DELETE []	8. INSERT(,) [Hi, how are you?]
	9. STOP [Hi, how are you?]

Table 3: Side-by-side obfuscation and restoration trajectory for a full continuation without a prefix.

In our primary experiments (insert+move+STOP action set at training and test time), obfuscation/restoration is applied only to y (the prompt prefix x is not edited), and we construct obfuscations using only deletions and moves, with restoration defined as the inverse mapping (each obfuscation deletion becomes an insertion of the deleted token, and each obfuscation move becomes the opposite move).

We generate an obfuscation trajectory by initializing the editable canvas to the continuation target and inserting the cursor at a uniformly random editable boundary, i.e., $C \leftarrow y$ and $u \sim \text{Uniform}(\{0, \dots, |y|\})$. At each obfuscation step, we sample an action from a simple state-dependent random policy with validity masking: with probability 0.8 we apply DELETE, which deletes the token immediately to the left of the cursor; with probability 0.2 we apply a move action MOVE(Δ), where Δ is sampled uniformly from the set of valid moves (i.e., those satisfying $u + \Delta \in \{0, \dots, \ell\}$). We store the resulting obfuscation action sequence as $B = (b_1, \dots, b_{n_{\text{obf}}})$, where n_{obf} is the number of obfuscation steps until termination. For each deletion step, we additionally record the identity of the deleted token in temporal order in an aligned side list $\mathcal{D} = (d_1, \dots, d_{n_{\text{del}}})$, where n_{del} is the number of deletions in B .

We represent cursor moves using a finite discrete set of displacements $\mathcal{M}$. In our implementation we use powers-of-two jumps up to a maximum displacement:

$$\mathcal{M} = \{ \pm 2^k \mid k \in \mathbb{Z}_{\geq 0}, 2^k \leq \text{max_move} \}.$$

At each step, we form the state-dependent valid subset $\mathcal{M}(C_t, u_t) = \{\Delta \in \mathcal{M} : u_t + \Delta \in \{0, \dots, \ell_t\}\}$ and sample Δ uniformly from $\mathcal{M}(C_t, u_t)$.

We terminate obfuscation when the editable canvas is empty (i.e., $\ell = 0$, leaving only the cursor boundary). If an obfuscation trajectory fails to reach $\ell = 0$ within a maximum step budget K_{max} ,

Algorithm 2 Obfuscation–restoration trajectory generation (primary implementation)

Require: Target tokens $y = (y_1, \dots, y_n)$, move set $\mathcal{M}$, max steps $K_{\max}$

- 1: Initialize canvas $C \leftarrow y$; initialize cursor $u \leftarrow \text{UNIFORM}(\{0, \dots, n\})$
 - 2: Initialize empty obfuscation action list $B \leftarrow ()$ and empty metadata list $\mathcal{D} \leftarrow ()$
 - 3: **for** $k = 1, 2, \dots, K_{\max}$ **do**
 - 4: **if** $|C| = 0$ **then break**
 - 5: **end if**
 - 6: Sample an obfuscation action $b_k \sim \pi_{\text{obf}}(\cdot \mid C, u)$ from DELETE or a move action $\text{MOVE}(\Delta)$, using validity masking
 - 7: **if** $b_k = \text{DELETE}$ **then**
 - 8: Let $x \leftarrow c_u$ be the token immediately left of the cursor boundary
 - 9: Delete x from C and update cursor $u \leftarrow u - 1$
 - 10: Append x to $\mathcal{D}$
 - 11: **else**
 - 12: Apply cursor move $u \leftarrow u + \Delta$
 - 13: **end if**
 - 14: Append b_k to B
 - 15: **end for**
 - 16: Let $n_{\text{obf}} \leftarrow |B|$
 - 17: Construct restoration actions $\star \leftarrow ()$ by iterating B from last to first:
 - 18: Treat $\mathcal{D}$ as a LIFO stack for deleted-token replay
 - 19: **for** $k = n_{\text{obf}}, n_{\text{obf}} - 1, \dots, 1$ **do**
 - 20: **if** $b_k = \text{DELETE}$ **then**
 - 21: pop x from $\mathcal{D}$ and append $\text{INSERT}(x)$ to $\star$
 - 22: **else**
 - 23: append $\text{MOVE}(-\Delta)$ to $\star$
 - 24: **end if**
 - 25: **end for**
 - 26: Append STOP to $\star$; **return** obfuscated start state and restoration trajectory $\star$
-

we discard the sample and resample a new trajectory.

Given an obfuscation trajectory $B = (b_1, \dots, b_{n_{\text{obf}}})$ and deleted-token metadata $\mathcal{D} = (d_1, \dots, d_{n_{\text{del}}})$, we construct a restoration trajectory $\star = (a_1, \dots, a_T)$ by scanning B from last to first and replacing each obfuscation action with its inverse restoration action. We treat $\mathcal{D}$ as a stack in temporal order and, whenever the reversed scan encounters a DELETE, we pop the most recently deleted token from $\mathcal{D}$ (LIFO order). For each $k = n_{\text{obf}}, n_{\text{obf}} - 1, \dots, 1$, we emit a restoration action

$$a \leftarrow \begin{cases} \text{INSERT}(x) \text{ where } x = \text{pop}(\mathcal{D}) & \text{if } b_k = \text{DELETE}, \\ \text{MOVE}(-\Delta) & \text{if } b_k = \text{MOVE}(\Delta), \end{cases}$$

and append these emitted actions in the scan order to form $\star$. Finally, we append STOP after the last restorative edit. In this strict-inverse construction, we produce one restoration action per obfuscation action, hence $T_{\text{rest}} = n_{\text{obf}} + 1$. Executing $\star$ from a blank canvas state deterministically reconstructs the target sequence and cursor position.

This reverse-inverse construction is a simple but effective supervision source. Reversing the obfuscation trajectory yields restoration actions that are guaranteed to be state-consistent with the executor dynamics, so each training target is a valid next edit for the current canvas/cursor state

rather than a synthetic label detached from state. In practice, this generates dense edit-history data that teaches the model to repair a corrupted canvas into a coherent sequence using the same action interface used at inference, which we find produces strong generation quality and robust non-monotonic editing behavior.

Instead of strict inverse mapping, restoration trajectories can be constructed by (i) an oracle policy, (ii) alignment/diff procedures, (iii) dynamic programming, or (iv) constrained search; and one can also use human text-edit traces, aggregate trajectories under the learned policy (Dagger), or fine-tune with RL. We discuss these variants in the Appendix (Appendices A.4 and A.5).

Because the model’s actions affect future states, naive supervised learning can suffer from compounding error. We consider two standard remedies: (i) dataset aggregation (Dagger-style), where we roll out the learned policy and label visited states with an oracle action (Ross et al., 2011), and (ii) reinforcement learning fine-tuning (e.g., PPO-style), where rewards are defined over completed trajectories (Schulman et al., 2017). In the reported experiments, we use supervised learning only (teacher forcing on restoration trajectories), without Dagger or RL fine-tuning (see Appendix Appendices A.4 and A.5).

Limitations of synthetic restoration trajectories. Our training data is constructed by applying random obfuscation trajectories and training the model to invert them. While this provides a simple and scalable source of supervision, these trajectories do not reflect the structure of human-like editing behavior. In particular, human edits are typically purposeful and context-dependent, involving targeted insertions, deletions, and refinements rather than random perturbations. As a result, the learned editing policies may differ from those that would arise from training on naturally occurring edit sequences. We expect that training on datasets of real edits, such as document revision histories or code editing traces, could yield more efficient and semantically meaningful editing strategies.

8 Experiments

8.1 Setup

We evaluate Reviser on a C4 continuation benchmark built from the English validation split of `allenai/c4`, comparing against autoregressive baselines at matched scales and non-autoregressive diffusion baselines. We pre-filter examples to total GPT-2 token length 144 to 216, use a 35-token prefix, and evaluate continuation quality toward a 180-token total sequence target.

We report results for two Reviser checkpoints: 100M and 300M. For the SEDD/MDLM comparison, we evaluate three seeds (123, 124, 125) with 1000 prompts per seed (3000 total per model). All main-text results use decoding with a maximum of 256 actions. For the diffusion baselines, both SEDD and MDLM are decoded with 128 diffusion steps. For models we train ourselves, we use matched 30B-token budgets: the trained AR baselines are trained on 30B text tokens, and Reviser models are trained on 30B edit-history tokens (including MOVE tokens). Because about 20% of Reviser training actions are MOVE actions, Reviser is exposed to fewer word tokens than the AR models under this matched token-budget accounting.

We report two metric families. First, we run pairwise 1v1 with-input arenas where the judge sees the prompt and both candidate continuations and selects a winner. For each arena example, we randomize candidate order (which model is shown as A vs. B) before constructing the judge input. We use Skywork-Critic-8B (Skywork AI, 2024) as the judge throughout all arena comparisons. Sec-

AR Baseline	Reviser WR $\uparrow$	AR WR $\uparrow$
AR Baseline 100M	61.30%	38.70%
AR Baseline 300M	54.40%	45.60%

Table 4: Reviser vs. AR baseline at 100M and 300M: arena win rates.

Model	evalPPL GPT-2 Large $\downarrow$	evalPPL Dream 7B $\downarrow$
AR Baseline 100M	13.52	13.96
Reviser 100M	30.64	22.67
AR Baseline 300M	12.51	12.02
Reviser 300M	17.77	15.77

Table 5: Reviser vs. AR baseline at 100M and 300M: evalPPL results (C4 decoding). Lower is better.

ond, evalPPL is computed on continuation tokens (lower is better), using two evaluator backbones: an autoregressive evaluator (GPT-2 Large) and a diffusion-style evaluator (Dream-7B). Reporting both reduces evaluator-family bias for Reviser’s nonstandard edit-action generation interface. We do not report regular teacher-forced PPL over Reviser action sequences because restoration trajectories are randomized and non-unique for a given final continuation, so trajectory-level likelihood is not a canonical, directly comparable quantity. Unless otherwise noted, all benchmark and arena results in this section are computed on 3000 samples. For reproducibility, code/configs/results are available at <https://github.com/Sean-Diab/Reviser>, and released Reviser checkpoints are available at <https://huggingface.co/sean-diab/reviser-checkpoints>.

8.2 Reviser vs. AR Baseline (100M and 300M)

We compare Reviser against autoregressive transformers of identical architecture trained on FineWeb data (the same distribution used to train Reviser) at both 100M and 300M scales. Throughout this section, we refer to these models as the AR baseline at each scale. This is a challenging setting: the AR model is on its home data and optimises the exact objective that evalPPL measures. Results are shown in Table 4. The judge prefers Reviser at both scales: 61.30% vs. 38.70% at 100M, and 54.40% vs. 45.60% at 300M.

Although Reviser has higher evalPPL than the AR baseline in Table 5, its arena results remain strong (Table 4). A likely reason is metric mismatch across generation paradigms: evalPPL is computed with an autoregressive scorer, while Reviser decodes via edit actions. The very high evalPPL values observed for SEDD/MDLM in Table 8 are consistent with this effect.

8.3 Reviser vs. AR Baselines

We evaluate Reviser in direct 1v1 arena matchups against a range of publicly available autoregressive models at roughly size-matched scales. All pairwise arena matchups in this subsection use 3000 samples per comparison.

At 100M scale, Reviser is preferred to all four AR baselines in this comparison, with especially large margins against Cerebras-GPT-111M and Pythia-160M. At 300M scale, Reviser remains competitive: it strongly outperforms Cerebras-GPT-256M and is approximately tied with Pythia-410M,

Model	100M Size	Rev WR $\uparrow$	AR WR $\uparrow$	300M Size	Rev WR $\uparrow$	AR WR $\uparrow$
Cerebras	111M	72.10%	27.90%	256M	71.30%	28.70%
Pythia	160M	68.80%	31.20%	410M	51.30%	48.70%
GPT-2	117M	58.60%	41.40%	345M	50.40%	49.60%
OPT	125M	53.90%	46.10%	350M	49.90%	50.10%

Table 6: Reviser vs. AR baselines at 100M and 300M scales: arena win rates under Skywork-Critic-8B.

Comparison	Reviser WR $\uparrow$	Baseline WR $\uparrow$
SEDD Small 169M	85.93%	14.07%
SEDD Absorb 353M	68.47%	31.53%
MDLM 170M	78.33%	21.67%

Table 7: Reviser vs. SEDD and MDLM: arena win rates.

GPT-2 Medium, and OPT-350M. Overall, these AR comparisons indicate a robust and promising profile.

8.4 Results: Reviser vs. SEDD and MDLM

Table 7 reports arena win rates against both diffusion baselines and shows that Reviser wins all three matchups, with its largest margin against SEDD Small 169M (85.93% vs. 14.07%), followed by MDLM 170M (78.33% vs. 21.67%), and SEDD Absorb 353M (68.47% vs. 31.53%). Table 8 reports the corresponding evalPPL values and shows the same qualitative pattern: Reviser is much lower than SEDD/MDLM under both evaluators at both available scales. MDLM does not have a comparable 300M checkpoint in our setup, so we omit MDLM-300M comparisons.

8.5 MAUVE

We also report MAUVE under a shared-3k protocol (Table 9): 3000 shared C4 examples, with references and model outputs truncated to 100 tokens, and BERT pseudo-loglikelihood features used for MAUVE computation. MAUVE measures distributional overlap between model outputs and references, with higher values indicating closer distributional match.

Table 9 shows that Reviser 300M achieves the strongest MAUVE in this comparison, while diffusion baselines (SEDD/MDLM) are substantially lower, consistent with the arena and evalPPL trends.

Model	evalPPL GPT-2 Large $\downarrow$	evalPPL Dream 7B $\downarrow$
Reviser 100M	30.64	22.67
SEDD Small 169M	158.72	117.05
MDLM 170M	144.69	94.30
Reviser 300M	17.77	15.77
SEDD Absorb 353M	98.68	83.28

Table 8: Reviser vs. SEDD and MDLM: evalPPL results (C4 decoding). Lower is better. MDLM 300M is omitted because no comparable checkpoint is available in this setup.

Model	MAUVE $\uparrow$
AR Baseline 100M	0.90
AR Baseline 300M	0.91
Reviser 300M	0.93
GPT-2 117M	0.88
GPT-2 Medium 345M	0.82
Reviser 100M	0.84
SEDD Absorb 353M	0.36
SEDD Small 169M	0.18
MDLM 170M	0.15

Table 9: Combined MAUVE results on the shared 3000-example C4 subset, using BERT pseudo-loglikelihood features and 100-token truncation for both references and model outputs. Higher is better.

Statistic (averaged across seeds)	100M	300M
Actions per output token	1.51	1.40
Insert fraction	67%	72%
Move fraction	33%	28%
Mean $ \Delta $ move distance	4.92	6.05
Move-distance mass on 1/2/4/8 moves (%)	40.5, 23.3, 15.8, 10.0	37.4, 22.7, 15.5, 11.1
Fraction backward moves	65.50%	64.50%
Mean insertion relative position	0.41	0.42
Fraction end-appends (cursor at end)	3.90%	3.10%
Fraction non end-appends	96.10%	96.90%
Examples with ≥ 1 backward revision	100%	100%

Table 10: Reviser trajectory diagnostics in decoding. “Backward revision” means at least one backward move followed by subsequent insertion into an earlier canvas region.

8.6 Trajectory Statistics

Table 10 shows that Reviser uses the edit interface in a strongly non-AR way. Move actions are frequent, backward moves dominate, and almost all insertions are non-end (mid-canvas) insertions. In every evaluated example, the trajectory contains at least one backward revision event, confirming that the model is not merely emulating a pure end-append decoder.

9 Limitations

Reviser is sequential in action space, so decoding is not fully parallelizable over final-token positions. As a result, throughput is still constrained by step-by-step generation, even though the model can revise non-monotonically.

Quality also depends on the supervision trajectories. Because training is based on synthetic obfuscation–restoration paths, performance can degrade under distribution shift and may exhibit compounding error when the model visits states that are weakly represented in training data.

The primary insert+move action set supports flexible revision, but some edits may require longer trajectories than richer operators (e.g., delete/replace). In addition, the history-only trunk must

maintain an implicit representation of the current canvas from the edit-history stream. In our reported runs, we did not observe this as a practical bottleneck, even for long trajectories, which is consistent with model capacity at 100M/300M. If needed, explicit state-conditioning mechanisms such as pooled canvas embeddings or cross-attention to canvas representations (Appendices A.2 and A.3) provide direct mitigation.

10 Future Work

The current results suggest several clear directions for improving Reviser. The main opportunities are to expand the action space beyond insert+move, strengthen state conditioning beyond the current history-only trunk, and move beyond fixed synthetic restoration trajectories toward training procedures that better reflect the model’s own inference-time distribution. More broadly, we view the present system as a proof of concept for autoregression over edit actions, and expect future variants to improve both output quality and editing efficiency while preserving the lightweight one-action decoding interface.

Learning beyond supervised trajectories. While we train Reviser using supervised restoration trajectories, an important next step is to move from synthetic trajectories to real edit supervision, e.g., human editing traces where people iteratively revise drafts, document revision histories, or code editing logs. The action-based interface is also compatible with more flexible training paradigms such as reinforcement learning or dataset aggregation (Dagger; see Appendix Appendices A.4 and A.5). In principle, any editing idea that can be represented as a token (or short token sequence) can be emitted by the transformer as an executable edit action, opening a broad and creative design space for richer supervision and editing behavior. Together, these approaches could allow the model to learn editing strategies under its own distribution, rather than following fixed trajectories. We leave empirical investigation of these directions to future work.

Flexible generation order as a potential advantage. A key property of Reviser is that it can build outputs in whatever order is useful for the task, rather than being constrained to final left-to-right token order. This gives the model an explicit draft-and-revise mechanism during generation: it can insert provisional content, move backward, and refine earlier regions before stopping. We hypothesize that, if learned reliably at scale, this flexibility may provide a meaningful quality advantage over strictly AR decoding, particularly in settings where iterative revision is important.

Structured editing and agent-based applications. Beyond text continuation, the cursor-action formulation also suggests applications to structured editing tasks such as code or document modification. Modern agent systems typically edit files by proposing diffs or patches over whole sequences (Yang et al., 2024; Gauthier, 2024); this can be inference-inefficient, since the model must spend tokens to emit a diff/patch command rather than directly editing the target content. In contrast, Reviser operates through localized cursor actions on a mutable canvas, which may provide a more direct interface for incremental editing. In such settings, richer edit operators such as DELETE, REPLACE, or span-level edits would likely make editing substantially more efficient by allowing the model to modify existing content directly rather than simulating corrections through longer insert-and-move trajectories. This structural alignment suggests the potential for more efficient editing workflows.

11 Conclusion

Reviser shows that a simple autoregressive-over-edits decoder, seeded with a deterministic prefix-seeding history and then decoding continuations via cursor actions, can produce genuinely non-monotonic text generation with a lightweight next-action interface. Across our experiments, the model performs frequent backward moves and mid-canvas insertions, is strongly preferred to the diffusion baselines we tested, and is competitive with roughly size-matched AR baselines. Overall, these results provide a strong example that edit-history autoregression is a practical and scalable path to models that can actively revise and improve their responses during generation.

References

- J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 2021.
- T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- P. Gauthier. Aider: Ai pair programming in your terminal, 2024. GitHub repository, <https://github.com/Aider-AI/aider>.
- M. Ghazvininejad, O. Levy, Y. Liu, and L. Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing*, 2019.
- J. Gu, J. Bradbury, C. Xiong, V. O. K. Li, and R. Socher. Non-autoregressive neural machine translation. In *International Conference on Learning Representations*, 2018.
- J. Gu, C. Wang, and J. Zhao. Levenshtein transformer. In *Advances in Neural Information Processing Systems*, 2019.
- J. Lee, E. Mansimov, and K. Cho. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In *Conference on Empirical Methods in Natural Language Processing*, 2018.
- X. L. Li, J. Thickstun, I. Gulrajani, P. S. Liang, and T. B. Hashimoto. Diffusion-lm improves controllable text generation. *Advances in Neural Information Processing Systems*, 2022.
- A. Lou, C. Meng, and S. Ermon. Sedd: Score-entropy discrete diffusion for language modeling. *arXiv preprint arXiv:2402.08667*, 2024.
- X. Ma, C. Zhou, X. Li, G. Neubig, and E. Hovy. Flowseq: Non-autoregressive conditional sequence generation with generative flow. In *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing*, 2019.
- J. Mallinson, A. Severyn, E. Malmi, S. Rothe, and S. Krause. Felix: Flexible text editing through tagging and insertion. In *Findings of the Association for Computational Linguistics: EMNLP*, 2020.
- E. Malmi, S. Krause, S. Rothe, K. Mirylenka, and A. Severyn. Encode, tag, realize: High-precision text editing. In *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing*, 2019.

- S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics*, 2011.
- S. Sahoo et al. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Skywork AI. Skywork-Critic: A strong critic model for reward modeling, 2024. *arXiv preprint arXiv:2410.09803*.
- M. Stern, W. Chan, J. Kiros, and J. Uszkoreit. Insertion transformer: Flexible sequence generation via insertion operations. In *International Conference on Machine Learning*, 2019.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le. Xlnet: Generalized autoregressive pretraining for language understanding. In *Advances in Neural Information Processing Systems*, 2019.

Contents

1	Introduction	1
2	Problem Setup and Design Goals	3
3	Compute Tradeoffs in Existing NAR and Edit-Based Transformers	3
4	Related Work	6
5	Reviser Formalism: State, Actions, and Executor	6
5.1	State and Canvas Representation	7
5.2	Action Space (Primary Implementation)	7
5.3	Executor: Canvas Update Mathematics	8
5.4	Validity Masking	8
6	Model	8
6.1	Edit-History Transformer	8
6.2	Generation Algorithm	9

7	Learning and Training	9
7.1	Worked Example: Edit History Trajectory	9
7.2	Obfuscation–Restoration Supervision (Main Training Procedure)	10
8	Experiments	13
8.1	Setup	13
8.2	Reviser vs. AR Baseline (100M and 300M)	14
8.3	Reviser vs. AR Baselines	14
8.4	Results: Reviser vs. SEDD and MDLM	15
8.5	MAUVE	15
8.6	Trajectory Statistics	16
9	Limitations	16
10	Future Work	17
11	Conclusion	18
A	Additional Reviser Variants	21
A.1	Other Edit Operators	21
A.2	Conditioning on the Canvas Representations via Pooled Canvas Embeddings	22
A.3	Cross-Attention to Canvas Representations	23
A.4	Dagger with an Alignment-Based Oracle	23
A.5	Reinforcement Learning Fine-Tuning	23
B	Prior Work Mechanisms and FLOPs-Based Efficiency Accounting	24
B.1	Goal and definitions (FLOPs-based, shared constants across models)	24
B.2	Primitive FLOPs functions	24
C	Model-by-model FLOPs accounting	25
C.1	Group 1. Anchors	25
C.2	Group 2. Diffusion NAR LMs (100M)	26
C.3	Group 3: Large-Scale Diffusion NAR LMs	28
C.4	Group 4. Refinement NAR LMs (MT)	31
D	More Qualitative Examples	34
D.1	Obfuscation–Restoration Trajectory Example	34
E	Selected Ranked Responses and Restoration Trajectories	34
E.1	100M Examples	34
E.2	300M Examples	39
F	Reproducibility Checklist	43

F.1	Artifacts, data, and runtime	43
F.2	End-to-end code path	43
F.3	Model and training	43
F.4	Hardware and FLOPs accounting	44
F.5	Decoding and inference configuration	44
F.6	Evaluation protocol and reporting	44

A Additional Reviser Variants

A.1 Other Edit Operators

This appendix subsection lists additional edit operators that can be supported by the same deterministic executor and validity masking used for INSERT/MOVE/STOP. More generally, any editing idea that can be represented by a token can be implemented as an action token (or a short composition of action tokens) and emitted by the transformer, provided the executor semantics are defined and validity-masked.

Replace the token immediately to the left of the cursor boundary with a new token $x \in V_A$:

$$\text{REPLACE}(x) : C_{t+1} = (c_{t,1}, \dots, c_{t,u_t-1}, x, c_{t,u_t+1}, \dots, c_{t,\ell_t}), \quad u_{t+1} = u_t.$$

Validity constraint: $u_t \in \{1, \dots, \ell_t\}$. (Equivalently, REPLACE(x) can be viewed as DELETE followed by INSERT(x) at boundary $u_t - 1$, but we include it as a single macro-action.)

For a span length $k \in \{1, \dots, u_t\}$, delete the last k tokens immediately to the left of the cursor boundary:

$$\text{SPANDELETE}(k) : C_{t+1} = (c_{t,1}, \dots, c_{t,u_t-k}, c_{t,u_t+1}, \dots, c_{t,\ell_t}), \quad u_{t+1} = u_t - k.$$

Validity constraint: $1 \leq k \leq u_t$.

Swap the last two tokens immediately to the left of the cursor boundary:

$$\text{SWAP} : C_{t+1} = (c_{t,1}, \dots, c_{t,u_t-2}, c_{t,u_t}, c_{t,u_t-1}, c_{t,u_t+1}, \dots, c_{t,\ell_t}), \quad u_{t+1} = u_t.$$

Validity constraint: $u_t \in \{2, \dots, \ell_t\}$.

Move a contiguous span of length k that ends at the token immediately left of the cursor boundary (i.e., indices $u_t - k + 1, \dots, u_t$) to a destination boundary j in the pre-removal canvas:

$$\begin{aligned} \text{SPANMOVE}(k, j) : C_{t+1} = & \\ & \begin{cases} (\underbrace{c_{t,1}, \dots, c_{t,j}}_{\text{left context}}, \underbrace{c_{t,u_t-k+1}, \dots, c_{t,u_t}}_{\text{moved span}}, \underbrace{c_{t,j+1}, \dots, c_{t,u_t-k}}_{\text{boosted context}}, \underbrace{c_{t,u_t+1}, \dots, c_{t,\ell_t}}_{\text{right context}}), & \text{if } 1 \leq j \leq u_t - k, \\ (\underbrace{c_{t,1}, \dots, c_{t,u_t-k}}_{\text{left context}}, \underbrace{c_{t,u_t+1}, \dots, c_{t,j}}_{\text{collapsed context}}, \underbrace{c_{t,u_t-k+1}, \dots, c_{t,u_t}}_{\text{moved span}}, \underbrace{c_{t,j+1}, \dots, c_{t,\ell_t}}_{\text{right context}}), & \text{if } u_t \leq j \leq \ell_t. \end{cases} \\ & u_{t+1} = \begin{cases} u_t, & \text{if } 1 \leq j \leq u_t - k, \\ j, & \text{if } u_t \leq j \leq \ell_t. \end{cases} \end{aligned}$$

Validity constraints: $1 \leq k \leq u_t$ and $j \in \{1, \dots, u_t - k\} \cup \{u_t, \dots, \ell_t\}$ (enforced by masking). Underbraced segments may be empty.

Copy (without removing) the length- k span immediately left of the cursor boundary and insert it at boundary $j \in \{0, \dots, \ell_t\}$:

$$\text{SPANCOPY}(k, j) : \quad C_{t+1} = \underbrace{(c_{t,1}, \dots, c_{t,j})}_{\text{left context}}, \underbrace{(c_{t,u_t-k+1}, \dots, c_{t,u_t})}_{\text{copied context}}, \underbrace{(c_{t,j+1}, \dots, c_{t,\ell_t})}_{\text{right context}}, \quad u_{t+1} = j + k,$$

with validity constraint $1 \leq k \leq u_t$. For both SPANCOPY and SPANMOVE, one can analogously copy/move spans taken from the right side of the cursor; we omit that symmetric math here.

All of the operators above admit straightforward supervised training using obfuscation restoration trajectories. The key observation is that each edit operator has a natural inverse. When temporary obfuscation-only insertions are used and later deleted, those transient insertions are treated as latent noise and excluded from the training loss because they are often random artifacts rather than meaningful target-response content.

A.2 Conditioning on the Canvas Representations via Pooled Canvas Embeddings

This subsection describes a variant in which action selection uses an explicit pooled summary of the current canvas. The motivation is simple: in a history-only trunk, canvas content is visible only indirectly through past actions and masking; adding a pooled canvas summary provides a direct channel for the model to “see” the current canvas state when choosing the next action. Our reported experiments use a history-only trunk without this module.

Attention pooling is just one instantiation of a broader idea: collapse variable-length canvas information into a fixed-size per-step vector (or small set of vectors) that can condition next-action prediction. There are many ways to perform this collapse (e.g., learned pooling queries, mean/max pooling with projections, convolutional pooling, recurrent summarizers, sparse/selective pooling, or learned routers). We use attention pooling here as a simple reference design.

Embed canvas tokens with the same shared token embedding table E as action tokens and a dedicated canvas positional table $P^{(C)}$. For the current canvas $C_t = (c_{t,1}, \dots, c_{t,\ell_t})$, define

$$z_i = E[c_{t,i}] + P^{(C)}[i], \quad i = 1, \dots, \ell_t.$$

Let $q \in \mathbb{R}^d$ be a learned query vector. Compute attention weights:

$$s_i = q^\top z_i, \quad \alpha_i = \frac{\exp(s_i)}{\sum_{j=1}^{\ell_t} \exp(s_j)}.$$

The pooled canvas embedding is:

$$\mu_t = \sum_{i=1}^{\ell_t} \alpha_i z_i \in \mathbb{R}^d.$$

In this variant, one injects the pooled canvas summary as an additive conditioning term on history-token inputs:

$$x_\tau = e_\tau + \mu_t, \quad \tau = 1, \dots, t-1.$$

One may instead apply a learned linear projection $W_m \in \mathbb{R}^{d \times d}$:

$$x_\tau = e_\tau + W_m \mu_t,$$

including the identity (no projection) as the simplest choice.

A.3 Cross-Attention to Canvas Representations

Another optional design lets the history stream cross-attend to the full canvas sequence (often paired with or used instead of a single pooled canvas vector; Appendix A.2). Let $H_t \in \mathbb{R}^{m \times d}$ denote the history-stream hidden states at step t (the model’s input token stream), and let $\mu_t \in \mathbb{R}^d$ be the pooled canvas embedding from Appendix A.2. Define $\mathbf{M}_t \in \mathbb{R}^{n \times d}$ as the pooled-canvas matrix whose rows are all equal to $\mu_t^\top$. In a cross-attention block, we compute queries from the history and keys/values from this pooled-canvas matrix:

$$Q_t = H_t W_Q, \quad K_t = \mathbf{M}_t W_K, \quad V_t = \mathbf{M}_t W_V,$$

and add the resulting cross-attention output as a residual update:

$$\tilde{H}_t = H_t + \text{SOFTMAX}\left(\frac{Q_t K_t^\top}{\sqrt{d_k}} + M\right) V_t.$$

In our main implementation, cross-attention is inserted after the history self-attention (and before the MLP) in each transformer layer, though in practice it often suffices to apply it only in the last L_{ca} layers to reduce cost. The cross-attention update can also be gated:

$$\tilde{H}_t = H_t + \lambda_{\text{ca}} \cdot \text{SOFTMAX}\left(\frac{Q_t K_t^\top}{\sqrt{d_k}} + M\right) V_t,$$

with either a learned scalar λ_{ca} per layer or a small gating MLP. With this pooled-canvas matrix, the block uses a compressed canvas memory derived from μ_t rather than per-token canvas states; in practice one can set $n = 1$ for minimal cost or use small n for implementation convenience.

A.4 DAgger with an Alignment-Based Oracle

Supervised training on fixed restoration trajectories can lead to distribution shift at inference time. DAgger (Ross et al., 2011) mitigates this by iterating between rolling out the current policy and retraining on oracle-labeled states from those rollouts.

Our oracle aligns the current canvas C_t against the target C^* via LCS-style sequence alignment, labeling tokens as matched (anchors) or mismatched. From this alignment it constructs a deterministic edit script: when the cursor is in a matched region, the oracle emits MOVE actions to route to mismatched regions; when the cursor is in a mismatched region, the oracle emits local content edits (e.g., INSERT and, when available, DELETE). DAgger then aggregates oracle-labeled states into the training set and re-optimizes by teacher forcing:

$$\min_{\theta} \mathbb{E}_{(s, a^*) \sim \mathcal{D}} \left[-\log p_{\theta}(a^* \mid s) \right].$$

A mixture policy that occasionally defers to the oracle prevents catastrophic divergence during early iterations, with the oracle mixing probability decayed over training so the policy gradually assumes full control.

A.5 Reinforcement Learning Fine-Tuning

While supervised restoration provides strong local imitation signals, it does not directly optimize sequence-level objectives such as holistic response quality, preference alignment, or length/compute

trade-offs. RL fine-tuning is a natural extension for optimizing such non-decomposable rewards over complete edit trajectories.

Editing is formalized as an episodic MDP: states s_t encode (C_t, u_t, H_t) plus a validity mask; actions are the edit operators; transitions are deterministic via the executor. Terminal rewards come from an LLM judge that ranks K candidate outputs and assigns a linearly decayed preference reward $R_{\text{pref}}(A^{(\pi(r))}) = 1 - (r - 1)/(K - 1)$.

Policy optimization uses PPO-style clipping with entropy regularization, an optional KL penalty to the supervised reference policy, and an optional action-type regularizer:

$$\mathcal{L}(\theta, \phi) = -\mathcal{L}_{\text{policy}}(\theta) + c_v \mathcal{L}_{\text{value}}(\phi) - c_e \mathcal{L}_{\text{ent}}(\theta) + c_{\text{kl}} \mathcal{L}_{\text{kl}}(\theta) + c_{\text{type}} \mathcal{L}_{\text{type}}(\theta).$$

To reduce early-rollout collapse, rollouts are warm-started with a decaying prefix of oracle actions. Extensions to RLHF-style preference optimization or multi-objective length/compute penalties are straightforward within this MDP formulation.

B Prior Work Mechanisms and FLOPs-Based Efficiency Accounting

B.1 Goal and definitions (FLOPs-based, shared constants across models)

We quantify inefficiency by comparing inference FLOPs to produce a full length- n output under each method’s decoding procedure, relative to an autoregressive (AR) baseline.

We fix the same backbone hyperparameters for every method (so ratios are meaningful):

$$n = 128, \quad |V_c| = 50,000, \quad d = d_{\text{model}} = 768, \quad L = 12, \quad d_{\text{ff}} = 4d, \quad K_{\text{max}} = 16.$$

For Reviser, the action vocabulary size is $|V_a| = |V_c| + |\mathcal{M}| + 1$; since $|\mathcal{M}| \ll |V_c|$, we often use $|V_a| \approx |V_c|$ when it does not materially affect ratios.

We count dominant matrix-multiply compute and omit small terms (biases, layernorm, activations, elementwise ops). All FLOPs expressions in this appendix are built from primitives in Appendix B.2. We count one fused multiply-accumulate (MAC) as one unit in $F_{\text{mult}}(a, b, c) = abc$. (To convert to conventions where one MAC equals two FLOPs, multiply all reported $F(\cdot)$ values by 2; ratios are unchanged.)

For each method $\mathcal{M}$, define $F_{\text{infer}}^{\mathcal{M}}(n)$ as the total FLOPs required to output an entire length- n sequence under that method’s inference procedure (summing all model calls, scoring/rounding steps, termination checks, etc.). We define inefficiency as $I_{\text{infer}}^{\mathcal{M}} \triangleq \frac{F_{\text{infer}}^{\mathcal{M}}(n)}{F_{\text{infer}}^{\text{AR}}(n)}$.

B.2 Primitive FLOPs functions

Let $A \in \mathbb{R}^{a \times b}$ and $B \in \mathbb{R}^{b \times c}$. We define $F_{\text{mult}}(a, b, c) \triangleq abc$, i.e., one multiply-accumulate (MAC) is one unit of compute.

We count dominant matrix-multiply compute and omit small terms (biases, layernorm, activations, elementwise ops). Here L denotes Transformer depth (number of layers), while n denotes sequence length. Because attention structure differs across settings, we distinguish full (bidirectional) attention from causal (lower-triangular) attention.

Define $F_{\text{transformer}}^{\text{full}}(n)$ as the dominant FLOPs for one forward pass through an L -block Transformer on a length- n sequence with full self-attention:

$$F_{\text{transformer}}^{\text{full}}(n) \triangleq L \left(4 F_{\text{mult}}(n, d, d) + 2 F_{\text{mult}}(n, n, d) + 2 F_{\text{mult}}(n, d, d_{\text{ff}}) \right).$$

With $d_{\text{ff}} = 4d$:

$$F_{\text{transformer}}^{\text{full}}(n) = L (12nd^2 + 2n^2d).$$

Causal self-attention uses only the $n(n+1)/2$ lower-triangular query-key pairs. We therefore model the attention-score and attention-apply matmuls by replacing the full n^2 pair count with $n(n+1)/2$. Define the dominant FLOPs for one forward pass with causal self-attention as

$$F_{\text{transformer}}^{\text{causal}}(n) \triangleq L \left(4 F_{\text{mult}}(n, d, d) + n(n+1)d + 2 F_{\text{mult}}(n, d, d_{\text{ff}}) \right),$$

where $n(n+1)d$ comes from summing the triangular costs of $QK^\top$ and $\text{Attn} \cdot V$, each equal to $\frac{n(n+1)}{2}d$. With $d_{\text{ff}} = 4d$:

$$F_{\text{transformer}}^{\text{causal}}(n) = L (12nd^2 + n^2d + nd).$$

Under this triangular convention, a causal forward pass at length n (computed in parallel under a causal mask) and KV-cached autoregressive decoding up to token n have the same dominant attention-matmul count.

Encoder self-attention is bidirectional, so we use the full-attention expression $F_{\text{enc}}(n) \triangleq F_{\text{transformer}}^{\text{full}}(n)$.

We model one decoder cross-attention layer (dominant matmuls) between a target length- n sequence and a source length- n_{src} sequence as:

$$F_{\text{xattn,layer}}(n, n_{\text{src}}) \triangleq \underbrace{F_{\text{mult}}(n, d, d)}_{Q \text{ proj (tgt)}} + \underbrace{2F_{\text{mult}}(n_{\text{src}}, d, d)}_{K, V \text{ proj (src)}} + \underbrace{F_{\text{mult}}(n, d, d)}_{O \text{ proj (tgt)}} + \underbrace{F_{\text{mult}}(n, d, n_{\text{src}})}_{\text{attn logits } (QK^\top)} + \underbrace{F_{\text{mult}}(n, n_{\text{src}}, d)}_{\text{attn apply } (\text{Attn} \cdot V)}.$$

Stacking across L decoder layers gives $F_{\text{xattn}}(n, n_{\text{src}}) \triangleq L F_{\text{xattn,layer}}(n, n_{\text{src}})$.

We model a dense vocab projection over m positions as $F_{\text{vocab}}(m) \triangleq F_{\text{mult}}(m, d, |V_c|)$.

C Model-by-model FLOPs accounting

C.1 Group 1. Anchors

C.1.1 Autoregressive Transformer (AR baseline)

We model the total dominant matmul compute of KV-cached AR decoding up to length n using the causal (triangular) attention convention. Across n decoding steps, the vocab head is applied once per generated token (total n applications), yielding:

$$F_{\text{infer}}^{\text{AR}}(n) \triangleq F_{\text{transformer}}^{\text{causal}}(n) + F_{\text{vocab}}(n).$$

Using $n = 128$, $|V_c| = 50,000$, $d = 768$, $L = 12$, and $d_{\text{ff}} = 4d$:

$$F_{\text{transformer}}^{\text{causal}}(128) \approx 11.02 \text{ G}, \quad F_{\text{vocab}}(128) \approx 4.92 \text{ G},$$

so

$$F_{\text{infer}}^{\text{AR}}(128) \approx 15.94 \text{ G}.$$

By definition, $I_{\text{infer}}^{\text{AR}} = 1$.

C.1.2 Cursor-style Edit Generation: Reviser (this work)

Reviser treats text generation as an editing process over a cursor position. At each step it predicts exactly one action (e.g., INSERT(token), or MOVE(Δ)), applies that edit to the current sequence, and continues iterating until termination. Unlike prior edit transformers that emit large per-position edit heads, Reviser emits a single action per step with a normal-sized token vocabulary head (plus a small set of edit actions), and it is trained with standard supervised learning (teacher forcing) in the same way as a typical Transformer.

Let p_{move} be the fraction of actions that are cursor-moves. To produce n inserted tokens, the expected number of action steps is $n_{\text{eff}} = \frac{n}{1-p_{\text{move}}}$. For $n = 128$, we report three settings: $p_{\text{move}} \in \{0.20, 0.28, 0.33\}$, giving $n_{\text{eff}} \in \{160, 177.78, 191.04\}$.

We approximate Reviser’s total decoding compute by evaluating the AR causal proxy at the effective length:

$$F_{\text{infer}}^{\text{Reviser}}(n) \triangleq F_{\text{transformer}}^{\text{causal}}(n_{\text{eff}}) + F_{\text{vocab}}(n_{\text{eff}}).$$

$$I_{\text{infer}}^{\text{Reviser}}(n) \triangleq \frac{F_{\text{infer}}^{\text{Reviser}}(n)}{F_{\text{infer}}^{\text{AR}}(n)}.$$

Using $F_{\text{infer}}^{\text{AR}}(128) \approx 15.94$ G, we obtain:

$$\begin{aligned} p_{\text{move}} = 0.20 : \quad & F_{\text{transformer}}^{\text{causal}}(160) \approx 13.83 \text{ G}, \quad F_{\text{vocab}}(160) \approx 6.14 \text{ G}, \\ & F_{\text{infer}}^{\text{Reviser}}(128) \approx 19.97 \text{ G}, \quad I_{\text{infer}}^{\text{Reviser}} \approx 1.25. \\ p_{\text{move}} = 0.28 : \quad & F_{\text{transformer}}^{\text{causal}}(177.78) \approx 15.39 \text{ G}, \quad F_{\text{vocab}}(177.78) \approx 6.83 \text{ G}, \\ & F_{\text{infer}}^{\text{Reviser}}(128) \approx 22.22 \text{ G}, \quad I_{\text{infer}}^{\text{Reviser}} \approx 1.39. \\ p_{\text{move}} = 0.33 : \quad & F_{\text{transformer}}^{\text{causal}}(191.04) \approx 16.56 \text{ G}, \quad F_{\text{vocab}}(191.04) \approx 7.34 \text{ G}, \\ & F_{\text{infer}}^{\text{Reviser}}(128) \approx 23.90 \text{ G}, \quad I_{\text{infer}}^{\text{Reviser}} \approx 1.50. \end{aligned}$$

C.2 Group 2. Diffusion NAR LMs (100M)

Many diffusion-style language models perform generation as T_{dec} repeated full-sequence Transformer forward passes, each followed by a vocabulary projection. For methods with this structure, we define the generation FLOPs as

$$F_{\text{diffusion}}^{\mathcal{M}_{\text{diff}}}(n, T_{\text{dec}}) \triangleq T_{\text{dec}} \left(F_{\text{transformer}}^{\text{full}}(n) + F_{\text{vocab}}(n) \right).$$

C.2.1 SEDD

Lou et al. (2024) parameterize SEDD, a reverse discrete diffusion process, and generate by running a fixed number T_{dec} of “network evaluations” (function evaluations) of Transformer conditioned on time and noise.

A conservative matmul proxy is that each step computes full-sequence hidden states (full attention) and produces categorical scores for all n positions:

$$F_{\text{infer}}^{\text{SEDD}}(n, T_{\text{dec}}) \triangleq F_{\text{diffusion}}^{\text{SEDD}}(n, T_{\text{dec}})$$

We report two representative step counts:

$$\begin{aligned} T_{\text{dec}} = 32 : \quad & F_{\text{infer}}^{\text{SEDD}}(128, 32) \approx 514.84 \text{ G}, \quad I_{\text{infer}}^{\text{SEDD}} = 32.30. \\ T_{\text{dec}} = 2048 : \quad & F_{\text{infer}}^{\text{SEDD}}(128, 2048) \approx 32,949.92 \text{ G}, \quad I_{\text{infer}}^{\text{SEDD}} = 2,067.25. \end{aligned}$$

C.2.2 MDLM

Sahoo et al. (2024) introduce MDLM, a masked discrete diffusion language model. Sampling starts from an all-[MASK] sequence and runs a discretized reverse diffusion with T_{dec} steps; at each step, the model predicts token distributions conditioned on the current partially denoised sequence.

Using the same conservative “full logits per step” proxy:

$$\begin{aligned} F_{\text{infer}}^{\text{MDLM}}(n, T_{\text{dec}}) &\triangleq F_{\text{diffusion}}^{\text{MDLM}}(n, T_{\text{dec}}) \\ F_{\text{infer}}^{\text{MDLM}}(128, 1000) &\approx 16,088.83 \text{ G}, \quad I_{\text{infer}}^{\text{MDLM}} = 1,009.40. \end{aligned}$$

C.2.3 D3PM

Austin et al. (2021) introduce D3PM, a general discrete denoising diffusion probabilistic model. In the text setting, it corresponds to a time-conditioned denoiser run for T_{dec} discrete reverse steps to transform a highly corrupted sequence into a clean sample.

$$\begin{aligned} F_{\text{infer}}^{\text{D3PM}}(n, T_{\text{dec}}) &\triangleq F_{\text{diffusion}}^{\text{D3PM}}(n, T_{\text{dec}}) \\ F_{\text{infer}}^{\text{D3PM}}(128, 1000) &\approx 16,088.83 \text{ G}, \quad I_{\text{infer}}^{\text{D3PM}} = 1,009.40. \end{aligned}$$

C.2.4 Diffusion-LM

Li et al. (2022) propose Diffusion-LM, which performs continuous diffusion over a length- n sequence of vectors and runs a (bidirectional) Transformer denoiser for T_{dec} reverse steps. To convert continuous vectors to discrete tokens, we upper-bound discretization/rounding by a dense vocab scoring matmul.

Let $|V|$ denote the (discrete) output vocabulary size for this model. We define

$$F_{\text{round}}(n) \triangleq F_{\text{mult}}(n, d, |V|),$$

(which equals $F_{\text{vocab}}(n)$ under our shared 100M configuration where $|V| = |V_c|$).

Each reverse step runs the denoiser once over the full length- n sequence (full attention):

$$F_{\text{denoise}}(n) \triangleq F_{\text{transformer}}^{\text{full}}(n).$$

The clamping trick optionally applies rounding to the predicted x_0 on a subset of the non-final reverse steps. Let $\gamma_{\text{clamp}} \in [0, 1]$ denote the fraction of the first $T_{\text{dec}} - 1$ steps on which clamping (rounding) is applied.

Method	Variant	$F_{\text{infer}}^{\mathcal{M}}$ (G)	$I_{\text{infer}}^{\mathcal{M}}$
AR (100M)	baseline	15.94	1
Reviser	$p_{\text{move}} \in \{0.20, 0.28, 0.33\}$	19.97–23.90	1.25–1.50
SEDD	$T_{\text{dec}}=32$ (full logits each step)	514.84	32.30
SEDD	$T_{\text{dec}}=2048$ (full logits each step)	32,949.92	2,067.25
MDLM	$T_{\text{dec}}=1000$ (ancestral; full logits each step)	16,088.83	1,009.40
D3PM	$T_{\text{dec}}=1000$ (discrete diffusion; full logits each step)	16,088.83	1,009.40
Diffusion-LM	$T_{\text{dec}}=200, \gamma_{\text{clamp}}=0$	2,239.64	140.51
Diffusion-LM	$T_{\text{dec}}=200, \gamma_{\text{clamp}}=1$	3,217.77	201.88
Diffusion-LM	$T_{\text{dec}}=2000, \gamma_{\text{clamp}}=0$	22,352.17	1,402.36

Table 11: 100M-tier diffusion-family FLOPs multipliers under a matmul-dominant proxy. (All rows use the shared 100M reference configuration.)

We pay T_{dec} denoiser evaluations, plus rounding once at the end, plus clamping-rounding on $\gamma_{\text{clamp}}(T_{\text{dec}} - 1)$ non-final steps:

$$F_{\text{infer}}^{\text{DiffLM}}(n) \triangleq T_{\text{dec}} F_{\text{denoise}}(n) + \left(1 + \gamma_{\text{clamp}}(T_{\text{dec}} - 1)\right) F_{\text{round}}(n).$$

Using $F_{\text{transformer}}^{\text{full}}(128) \approx 11.17$ G and $F_{\text{round}}(128) = F_{\text{mult}}(128, 768, 50000) \approx 4.92$ G, we obtain:

$$\begin{aligned} T_{\text{dec}} = 200, \gamma_{\text{clamp}} = 0 : \quad & F_{\text{infer}}^{\text{DiffLM}}(128) \approx 2,239.64 \text{ G}, \quad I_{\text{infer}}^{\text{DiffLM}} = 140.51. \\ T_{\text{dec}} = 200, \gamma_{\text{clamp}} = 1 : \quad & F_{\text{infer}}^{\text{DiffLM}}(128) \approx 3,217.77 \text{ G}, \quad I_{\text{infer}}^{\text{DiffLM}} = 201.88. \\ T_{\text{dec}} = 2000, \gamma_{\text{clamp}} = 0 : \quad & F_{\text{infer}}^{\text{DiffLM}}(128) \approx 22,352.17 \text{ G}, \quad I_{\text{infer}}^{\text{DiffLM}} = 1,402.36. \end{aligned}$$

C.2.5 Summary table (100M tier)

C.3 Group 3: Large-Scale Diffusion NAR LMs

The large backbones used in this subsection (e.g., LLaMA/Qwen families) commonly use (i) a gated MLP with three linear maps (`gate_proj`, `up_proj`, `down_proj`), and (ii) grouped-query attention (GQA) where K/V have fewer heads than Q. Let h be the number of query heads and h_{kv} the number of K/V heads. With head dimension $d_{\text{head}} = d/h$, define the effective K/V projection width $d_{\text{kv}} \triangleq h_{\text{kv}} d_{\text{head}}$. (For standard MHA, $h_{\text{kv}} = h$ so $d_{\text{kv}} = d$.)

For n positions, Q and O are $(n, d) \times (d, d)$, while K and V are $(n, d) \times (d, d_{\text{kv}})$:

$$F_{\text{proj}}^{\text{GQA}}(n; d, d_{\text{kv}}) \triangleq 2F_{\text{mult}}(n, d, d) + 2F_{\text{mult}}(n, d, d_{\text{kv}}).$$

A gated MLP uses three matmuls (gate, up, down):

$$F_{\text{mlp}}^{\text{gated}}(n; d, d_{\text{ff}}) \triangleq 3F_{\text{mult}}(n, d, d_{\text{ff}}).$$

$$F_{\text{transformer}}^{\text{full,gated}}(n; d, L, d_{\text{ff}}, d_{\text{kv}}) \triangleq L \left(F_{\text{proj}}^{\text{GQA}}(n; d, d_{\text{kv}}) + 2F_{\text{mult}}(n, n, d) + F_{\text{mlp}}^{\text{gated}}(n; d, d_{\text{ff}}) \right).$$

$$F_{\text{transformer}}^{\text{causal,gated}}(n; d, L, d_{\text{ff}}, d_{\text{kv}}) \triangleq L \left(F_{\text{proj}}^{\text{GQA}}(n; d, d_{\text{kv}}) + n(n+1)d + F_{\text{mlp}}^{\text{gated}}(n; d, d_{\text{ff}}) \right).$$

C.3.1 Large-scale reference configurations (native sizes)

We keep each large model at its native architecture and compare to a size-matched AR baseline of the same scale. All comparisons use the same output length $n = 128$ and the same FLOPs conventions as in Appendix B.2.

For a decoder-only Transformer, we denote the number of layers by L , hidden size by d , MLP width by d_{ff} , and vocabulary size by $|V|$. For attention, let h be the number of query heads and h_{kv} the number of K/V heads (Grouped-Query Attention, GQA). With head dimension $d_{\text{head}} \triangleq d/h$, the effective K/V projection width is $d_{\text{kv}} \triangleq h_{\text{kv}} d_{\text{head}}$. (For standard multi-head attention, $h_{\text{kv}} = h$ so $d_{\text{kv}} = d$.)

$$L = 64, \quad d = 5120, \quad d_{\text{ff}} = 27648, \quad |V| = 152064, \quad h = 40, \quad h_{\text{kv}} = 8.$$

Here $d_{\text{head}} = 5120/40 = 128$ and $d_{\text{kv}} = 8 \cdot 128 = 1024$.

$$L = 28, \quad d = 3584, \quad d_{\text{ff}} = 18944, \quad |V| = 152064, \quad h = 28, \quad h_{\text{kv}} = 4.$$

Here $d_{\text{head}} = 3584/28 = 128$ and $d_{\text{kv}} = 4 \cdot 128 = 512$.

For LLaDA-8B we use the published LLaDA config (full-attention masked-token predictor):

$$\text{LLaDA-8B: } L = 32, \quad d = 4096, \quad d_{\text{ff}} = 12288, \quad |V| = 126464, \quad h = 32, \quad h_{\text{kv}} = 32,$$

so $d_{\text{head}} = 4096/32 = 128$ and $d_{\text{kv}} = 32 \cdot 128 = 4096$ (no GQA reduction).

For the size-matched AR baseline we use LLaMA3-8B (causal decoder with GQA):

$$\text{LLaMA3-8B (AR): } L = 32, \quad d = 4096, \quad d_{\text{ff}} = 14336, \quad |V| = 128256, \quad h = 32, \quad h_{\text{kv}} = 8,$$

so $d_{\text{head}} = 128$ and $d_{\text{kv}} = 8 \cdot 128 = 1024$.

C.3.2 AR baselines (size-matched)

For each backbone, we use a LLaMA/Qwen-style decoder proxy (GQA + gated MLP):

$$F_{\text{infer}}^{\text{AR}}(n) \triangleq F_{\text{transformer}}^{\text{causal,gated}}(n; d, L, d_{\text{ff}}, d_{\text{kv}}) + F_{\text{mult}}(n, d, |V|).$$

$$F_{\text{infer}}^{\text{AR (Qwen2.5-32B)}}(128) \approx 4,099.39 \text{ G.}$$

$$F_{\text{infer}}^{\text{AR (Qwen2.5-7B)}}(128) \approx 906.65 \text{ G.}$$

$$F_{\text{infer}}^{\text{AR (LLaMA3-8B)}}(128) \approx 962.76 \text{ G.}$$

C.3.3 SDLM-32B-D4 (Sequential Diffusion Language Model)

SDLM predicts a fixed block of D tokens per forward pass, then selects the longest high-confidence prefix (length $k \in \{1, \dots, D\}$) and commits it. KV-cache states for committed tokens are reused. Here, D is the block size (we use $D = 4$), and $\bar{k} \triangleq \mathbb{E}[k]$ is the average committed tokens per pass.

To propose a block of D new tokens given a cached prefix of length n_{pre} , we count: (i) Q/K/V/O projections + gated MLP on the D new positions, and (ii) attention matmuls for D queries attending

to the cached prefix plus the causal within-block keys. Let d_{kv} be the K/V projection width (for GQA; for MHA, $d_{kv} = d$). We use:

$$F_{kvblk}(D, n_{pre}; d_{ff}, d_{kv}) \triangleq L \left(\underbrace{2F_{mult}(D, d, d) + 2F_{mult}(D, d, d_{kv})}_{\text{Q,O and K,V projections}} + \underbrace{3F_{mult}(D, d, d_{ff})}_{\text{gated MLP}} + \underbrace{2F_{mult}(D, n_{pre}, d) + F_{mult}(D, D+1, d)}_{\text{attn (prefix + causal within-block)}} \right).$$

With $S \approx \lceil n/\bar{k} \rceil$ passes and prefix length $n_{pre} \approx (s-1)\bar{k}$:

$$F_{infer}^{SDLM}(n) \triangleq \sum_{s=1}^S \left(F_{kvblk}(D, (s-1)\bar{k}; d_{ff}, d_{kv}) + F_{mult}(D, d, |V|) \right).$$

$$F_{infer}^{SDLM-32B-D4}(128) \approx 8,198.94 \text{ G}, \quad I_{infer}^{SDLM-32B-D4} = 2.00.$$

C.3.4 Dream-7B (diffusion LLM; Qwen2.5-7B backbone)

Dream performs masked discrete diffusion: starting from a fully masked response, it iteratively denoises for T_{dec} steps using a full-attention Transformer denoiser.

Let m_t be the number of masked positions scored at step t . For a simple closed form we approximate a linear schedule: $\sum_{t=1}^{T_{dec}} m_t \approx n \frac{T_{dec}+1}{2}$.

$$F_{infer}^{Dream}(n) \approx T_{dec} F_{transformer}^{full}(n) + \frac{T_{dec}+1}{2} F_{vocab}(n; |V|).$$

$$\begin{aligned} T_{dec} = 10 : \quad & F_{infer}^{Dream-7B}(128) \approx 8,768.93 \text{ G}, \quad I_{infer}^{Dream-7B} = 9.67. \\ T_{dec} = 20 : \quad & F_{infer}^{Dream-7B}(128) \approx 17,502.98 \text{ G}, \quad I_{infer}^{Dream-7B} = 19.31. \end{aligned}$$

C.3.5 LLaDA-8B (diffusion mask predictor; compared to LLaMA3-8B AR)

LLaDA predicts masked tokens with a full-attention Transformer (no causal mask) and runs a reverse masking/denoising process for T_{dec} steps. For instruct-style decoding, it can also generate left-to-right in blocks, applying the reverse process inside each block. Because this configuration uses $h_{kv}=h$ (no GQA reduction), its K/V-side projection terms are larger than in GQA AR baselines, which contributes to the larger compute multiplier.

$$F_{infer}^{LLaDA}(n) \approx T_{dec} F_{transformer}^{full}(n) + \frac{T_{dec}+1}{2} F_{vocab}(n; |V|).$$

Partition the response into $B = \lceil n/b \rceil$ blocks of size b , and run T_b reverse steps per block. We approximate full-attention trunk length as $n_j = \min(jb, n)$ for block j , and vocab logits only for the b block positions:

$$F_{infer,blk}^{LLaDA}(n) \triangleq \sum_{j=1}^B \left(T_b F_{transformer}^{full}(n_j) + \frac{T_b+1}{2} F_{vocab}(b; |V|) \right).$$

$$\begin{aligned} T_{dec} = 256 : \quad & F_{infer}^{LLaDA-8B}(128) \approx 238,317.94 \text{ G}, \quad I_{infer}^{LLaDA-8B} = 247.54. \\ \text{semi-AR: } b = 32, T_b = 16 : \quad & F_{infer,blk}^{LLaDA-8B}(128) \approx 36,426.56 \text{ G}, \quad I_{infer,blk}^{LLaDA-8B} = 37.84. \end{aligned}$$

Method	Variant	$F_{\text{infer}}^{\mathcal{M}}$ (G)	$I_{\text{infer}}^{\mathcal{M}}$
AR (Qwen2.5-32B)	baseline	4,099.39	1
SDLM-32B-D4	$D=4, \bar{k}=2$ (no verify)	8,198.94	2.00
AR (Qwen2.5-7B)	baseline	906.65	1
Dream-7B	$T_{\text{dec}}=10$ (lin. unmask)	8,768.93	9.67
Dream-7B	$T_{\text{dec}}=20$ (lin. unmask)	17,502.98	19.31
AR (LLaMA3-8B)	baseline	962.76	1
LLaDA-8B	$T_{\text{dec}}=256$ (full-length)	238,317.94	247.54
LLaDA-8B	semi-AR remark: $b=32, T_b=16$	36,426.56	37.84

Table 12: Large-scale diffusion-family FLOPs multipliers under a matmul-dominant proxy. Each block uses a size-matched AR baseline (Qwen2.5-32B, Qwen2.5-7B, or LLaMA3-8B).

C.3.6 Summary table (large-scale)

C.4 Group 4. Refinement NAR LMs (MT)

C.4.1 Levenshtein Transformer (LevT)

Gu et al. (2019) describe the Levenshtein Transformer (LevT), which runs R refinement iterations. Each iteration applies three predictors in sequence: delete π^{del} , placeholder-count π^{plh} , and token-fill π^{tok} . This corresponds to three trunk evaluations per iteration, with optional early exit for the first two: $f_{\text{del}} = \frac{b_{\text{del}}}{L}$ and $f_{\text{plh}} = \frac{b_{\text{plh}}}{L}$, with $f_{\text{del}}, f_{\text{plh}} \in (0, 1]$.

We evaluate delete + placeholder predictors at an average length $n_0 \triangleq 0.8n$, and we evaluate the token-fill predictor at length n (after insertion returns to the target length). For the token-fill vocabulary head, we assume only placeholder positions require vocab logits, with $m \triangleq 0.4n$ denoting the estimated number of placeholder tokens to fill at each refinement iteration. This LevT accounting is a decoder-side iterative-edit proxy (matching the LevT edit cycle) rather than a full source-conditioned MT accounting with separate encoder/cross-attention terms.

LevT runs bidirectional/refinement-style passes, so we use full attention in the trunk terms:

$$\begin{aligned}
F_{\text{iter}}^{\text{LevT}}(n) \approx & \underbrace{f_{\text{del}} F_{\text{transformer}}^{\text{full}}(n_0)}_{\text{delete trunk}} + \underbrace{f_{\text{plh}} F_{\text{transformer}}^{\text{full}}(n_0)}_{\text{placeholder trunk}} + \underbrace{F_{\text{transformer}}^{\text{full}}(n)}_{\text{token trunk}} \\
& + \underbrace{F_{\text{mult}}(n_0, d, 2)}_{\text{delete head}} + \underbrace{F_{\text{mult}}(n_0 + 1, 2d, K_{\text{max}} + 1)}_{\text{placeholder-count head}} + \underbrace{F_{\text{vocab}}(m)}_{\text{vocab projection on placeholders}},
\end{aligned}$$

where $n_0 = 0.8n$ and $m = 0.4n$. These are average-case analytical surrogates and need not be integers. Total decoding cost is

$$F_{\text{infer}}^{\text{LevT}}(n) \approx R \cdot F_{\text{iter}}^{\text{LevT}}(n).$$

Using the shared reference configuration with $n = 128$, $n_0 = 102.4$, and $m = 51.2$:

$$F_{\text{transformer}}^{\text{full}}(102.4) \approx 8.89 \text{ G}, \quad F_{\text{transformer}}^{\text{full}}(128) \approx 11.17 \text{ G},$$

and head costs

$$F_{\text{mult}}(102.4, 768, 2) \approx 0.16 \text{ M}, \quad F_{\text{mult}}(103.4, 1536, 17) \approx 2.70 \text{ M}, \quad F_{\text{vocab}}(51.2) \approx 1.97 \text{ G}.$$

Set $f_{\text{del}} = f_{\text{plh}} = 1$. Then:

$$F_{\text{iter,noEE}}^{\text{LevT}}(128) \approx 30.92 \text{ G.}$$

With $R = 5$ and $R = 10$:

$$F_{\text{infer,noEE}}^{\text{LevT}}(128, R = 5) \approx 154.62 \text{ G,} \quad F_{\text{infer,noEE}}^{\text{LevT}}(128, R = 10) \approx 309.24 \text{ G.}$$

Set $f_{\text{del}} = f_{\text{plh}} = 0.5$. Then:

$$F_{\text{iter,EE}}^{\text{LevT}}(128) \approx 22.03 \text{ G.}$$

With $R = 5$ and $R = 10$:

$$F_{\text{infer,EE}}^{\text{LevT}}(128, R = 5) \approx 110.17 \text{ G,} \quad F_{\text{infer,EE}}^{\text{LevT}}(128, R = 10) \approx 220.33 \text{ G.}$$

Using $F_{\text{infer}}^{\text{AR}}(128) \approx 15.94 \text{ G}$:

$$I_{\text{infer,noEE}}^{\text{LevT}}(R = 5) = 9.70, \quad I_{\text{infer,noEE}}^{\text{LevT}}(R = 10) = 19.40,$$

$$I_{\text{infer,EE}}^{\text{LevT}}(R = 5) = 6.91, \quad I_{\text{infer,EE}}^{\text{LevT}}(R = 10) = 13.82.$$

C.4.2 Insertion-based Generation: Insertion Transformer (Stern et al., 2019)

Stern et al. (2019) propose the Insertion Transformer (InsT), which generates a sequence by iteratively inserting tokens into slots (gaps). At iteration t , the current partial sequence has length n_t . The model runs a Transformer pass on the current sequence and predicts, for each slot, either a vocabulary token to insert or a special **no-insert** action.

Insertion Transformer uses bidirectional passes over the current partial sequence, so we use full attention:

$$F_{\text{iter}}^{\text{InsT}}(n_t) \triangleq F_{\text{transformer}}^{\text{full}}(n_t) + F_{\text{vocab}}(n_t + 1),$$

where $n_t + 1$ slots are scored. (The extra **no-insert** class changes $|V_c|$ by +1, which is negligible at $|V_c| = 50\text{k}$.) AR baseline: $F_{\text{infer}}^{\text{AR}}(128) \approx 15.94 \text{ G}$. Take $n_t \in \{1, 2, 4, 8, 16, 32, 64, 128\}$. Then:

$$\sum_t F_{\text{transformer}}^{\text{full}}(n_t) \approx 22.06 \text{ G,} \quad \sum_t F_{\text{vocab}}(n_t + 1) \approx 10.10 \text{ G,}$$

so

$$F_{\text{infer,bal}}^{\text{InsT}}(128) \approx 32.16 \text{ G,} \quad I_{\text{infer,bal}}^{\text{InsT}} = 2.02.$$

Take $n_t \in \{1, 2, \dots, 128\}$. Then:

$$\sum_t F_{\text{transformer}}^{\text{full}}(n_t) \approx 714.26 \text{ G,} \quad \sum_t F_{\text{vocab}}(n_t + 1) \approx 321.95 \text{ G,}$$

so

$$F_{\text{infer,ser}}^{\text{InsT}}(128) \approx 1,036.20 \text{ G,} \quad I_{\text{infer,ser}}^{\text{InsT}} = 65.01.$$

C.4.3 One-shot NAT (fertility / length latents): Non-Autoregressive NMT (Gu et al., 2018)

Gu et al. (2018) propose one-shot non-autoregressive translation (NAT), which predicts all target tokens in parallel by using discrete fertility/length latents and a single encoder–decoder pass.

For this numerical instantiation we use $n_{\text{src}} = n = 128$. Let K_{fert} denote the number of discrete fertility classes (the maximum fertility bucketed by the model), so each source token predicts a fertility $f_i \in \{0, 1, \dots, K_{\text{fert}} - 1\}$; here we set $K_{\text{fert}} = 50$.

One-shot NAT runs a single encoder–decoder evaluation:

$$F_{\text{infer}}^{\text{NAT}}(n, n_{\text{src}}) \triangleq F_{\text{enc}}(n_{\text{src}}) + F_{\text{mult}}(n_{\text{src}}, d, K_{\text{fert}}) + \left(F_{\text{transformer}}^{\text{full}}(n) + F_{\text{xattn}}(n, n_{\text{src}}) \right) + F_{\text{vocab}}(n).$$

With $n = n_{\text{src}} = 128$:

$$F_{\text{xattn}}(128, 128) \approx 3.93 \text{ G}, \quad F_{\text{fert}}(128) = F_{\text{mult}}(128, 768, 50) \approx 4.92 \text{ M}.$$

Therefore:

$$F_{\text{infer}}^{\text{NAT}}(128, 128) \approx 31.19 \text{ G}, \quad I_{\text{infer}}^{\text{NAT}} = 1.96.$$

C.4.4 Iterative Mask Refinement: Mask-Predict / CMLM (Ghazvininejad et al., 2019)

Ghazvininejad et al. (2019) start Mask-Predict from a fully-masked target and perform T_{mp} refinement cycles, each time masking low-confidence tokens and re-predicting them in parallel.

We instantiate $T_{\text{mp}} = 10$ and the common linear decay schedule:

$$m_0 = n, \quad m_t = \left\lfloor n \cdot \frac{T_{\text{mp}} - t}{T_{\text{mp}}} \right\rfloor \quad (t = 1, \dots, T_{\text{mp}} - 1).$$

For $n = 128$, $T_{\text{mp}} = 10$:

$$(m_0, \dots, m_9) = (128, 115, 102, 89, 76, 64, 51, 38, 25, 12), \quad \sum_{t=0}^9 m_t = 700.$$

At each iteration we pay decoder trunk + cross-attention + vocab projection on only the masked tokens:

$$F_{\text{iter}}^{\text{CMLM}}(t, n, n_{\text{src}}) = F_{\text{transformer}}^{\text{full}}(n) + F_{\text{xattn}}(n, n_{\text{src}}) + F_{\text{vocab}}(m_t).$$

Encoder cost is paid once: $F_{\text{enc}}(n_{\text{src}}) = F_{\text{transformer}}^{\text{full}}(n_{\text{src}})$.

With $n_{\text{src}} = n = 128$:

$$F_{\text{transformer}}^{\text{full}}(128) + F_{\text{xattn}}(128, 128) \approx 15.10 \text{ G}.$$

Masked-only vocab projection across all iterations:

$$\sum_{t=0}^{T_{\text{mp}}-1} F_{\text{vocab}}(m_t) \approx 26.88 \text{ G}.$$

Therefore:

$$F_{\text{infer}}^{\text{CMLM}}(128, 128) = F_{\text{enc}}(128) + \sum_{t=0}^{T_{\text{mp}}-1} F_{\text{iter}}^{\text{CMLM}}(t, 128, 128) \approx 189.05 \text{ G},$$

$$I_{\text{infer}}^{\text{CMLM}} = 11.86.$$

Method	Variant	$F_{\text{infer}}^{\mathcal{M}}$ (G)	$I_{\text{infer}}^{\mathcal{M}}$
AR	baseline	15.94	1
Reviser	$p_{\text{move}} \in \{0.20, 0.28, 0.33\}$	19.97–23.90	1.25–1.50
LevT	no EE, $R = 5$	154.62	9.70
LevT	no EE, $R = 10$	309.24	19.40
LevT	EE ($f_* = 0.5$), $R = 5$	110.17	6.91
LevT	EE ($f_* = 0.5$), $R = 10$	220.33	13.82
InsT	balanced-tree	32.16	2.02
InsT	serial	1,036.20	65.01
NAT	one-shot	31.19	1.96
CMLM	Mask-Predict, $T_{\text{mp}} = 10$	189.05	11.86

Table 13: Summary of inference compute under a matmul-dominant FLOPs proxy.

C.4.5 Summary table

D More Qualitative Examples

Table 14 gives a concrete obfuscation–restoration trajectory example used for qualitative inspection.

D.1 Obfuscation–Restoration Trajectory Example

The example referred to in this subsection is Table 14.

We illustrate how an obfuscation trajectory (left) can be inverted to form a restoration trajectory (right). The cursor is shown as a vertical bar “|” inside the bracketed canvas.

Concatenating the executed edit actions, the obfuscation trajectory is:

$$B = [\text{DELETE}, \text{MOVE}(-1), \text{DELETE}, \text{INSERT}(\text{hello}), \text{MOVE}(+2), \text{DELETE}, \text{DELETE}, \text{DELETE}, \text{MOVE}(+1), \text{DELETE}].$$

Reversing the order and inverting each action yields the restoration trajectory:

$$\star = [\text{INSERT}(\text{lamb.}), \text{MOVE}(-1), \text{INSERT}(\text{hello}), \text{INSERT}(\text{had}), \text{INSERT}(\text{little}), \text{MOVE}(-2), \text{DELETE}, \text{INSERT}(\text{Mary}), \text{MOVE}(+1), \text{INSERT}(\text{a}), \text{STOP}].$$

Applying $\star$ to the blank state deterministically reconstructs the original canvas shown on the left.

E Selected Ranked Responses and Restoration Trajectories

Interactive HTML visualizations for the examples in this section are available in the GitHub repository under [visualizations/](#).

E.1 100M Examples

100M Reviser Example 1

Text (prompt segment in blue).

Obfuscation (corrupt target $\rightarrow$ blank)	Restoration (invert & apply $\rightarrow$ target)
1. Initial [Mary had a little lamb.]	1. Initial []
2. DELETE [Mary had little lamb.]	2. INSERT(lamb.) [lamb.]
3. MOVE(-1) [Mary had little lamb.]	3. MOVE(-1) [lamb.]
4. DELETE [had little lamb.]	4. INSERT(hello) [hello lamb.]
5. INSERT(hello) [hello had little lamb.]	5. INSERT(had) [hello had lamb.]
6. MOVE(+2) [hello had little lamb.]	6. INSERT(little) [hello had little lamb.]
7. DELETE [hello had lamb.]	7. MOVE(-2) [hello had little lamb.]
8. DELETE [hello lamb.]	8. DELETE [had little lamb.]
9. DELETE [lamb.]	9. INSERT(Mary) [Mary had little lamb.]
10. MOVE(+1) [lamb.]	10. MOVE(+1) [Mary had little lamb.]
11. DELETE []	11. INSERT(a) [Mary had a little lamb.]
	12. STOP [Mary had a little lamb.]

Table 14: Side-by-side obfuscation and restoration (richer-action-space variant for illustration). Restoration is obtained by reversing the obfuscation actions and inverting each step (Delete $\leftrightarrow$ Insert of the deleted token, Move(Δ) $\leftrightarrow$ Move($-\Delta$), Insert(token) $\leftrightarrow$ Delete).

Claudine started ballet at the age of three in Chapel Hill, North Carolina and continued at Pofahl Studios under the instruction of Kim Tuttle and Judy Skinner in Gainesville, Texas. She taught at various studios including New York City Dance Center, the Brooklyn School of Dance, and the Center for Arts & Dance at Syracuse Dance Studio in Little Rock, New York. Claudine began ballet at the age of three. She started teaching at the same time, and continued teaching at the age of two teaching. She also taught with several dance classes during her time in the private and private classes. She has taught classes both in the private and semi private areas, She has taught for years as an educator and has had to have more than as many dancers have taught.

Restoration trajectory (189 actions).

INSERT `Cl', INSERT `aud', INSERT `ine', INSERT `started', INSERT `ballet', INSERT `at', INSERT `the', INSERT `age', INSERT `of', INSERT `three', INSERT `in', INSERT `Chapel', INSERT `Hill', INSERT `', INSERT `North', INSERT `Carolina', INSERT `and', INSERT `continued', INSERT `at', INSERT `P', INSERT `of', INSERT `ahl', INSERT `Studios', INSERT `under', INSERT `the', INSERT `instruction', INSERT `of', INSERT `Kim', INSERT `Tut', INSERT `tle', INSERT `and', INSERT `Judy', INSERT `Skinner', INSERT `in', INSERT `Gain', INSERT `.', [MOVE -1], INSERT `have', INSERT `taught', [MOVE -2], INSERT `as', INSERT `many', INSERT `dancers', [MOVE -2], [MOVE -1], INSERT `and', INSERT `has', INSERT `had', INSERT `to', [MOVE -4], INSERT `', [MOVE -1], INSERT `', INSERT `.', [MOVE -2], [MOVE +2], [MOVE -1], [MOVE -1], INSERT `

New', INSERT `York', INSERT `.', INSERT `\\n', INSERT `Cl', INSERT `aud', INSERT `ine', INSERT `began', [MOVE -8], INSERT `in', INSERT `Little', INSERT `Rock', INSERT `,', [MOVE +8], INSERT `ballet', INSERT `at', INSERT `She', INSERT `started', INSERT `teaching', INSERT `at', [MOVE -4], INSERT `the', INSERT `age', INSERT `of', INSERT `three', INSERT `.', [MOVE +4], INSERT `the', INSERT `same', INSERT `time', [MOVE +1], INSERT `and', INSERT `teaching', [MOVE +2], INSERT `as', [MOVE +4], [MOVE -8], INSERT `continued', [MOVE +4], INSERT `an', INSERT `educator', [MOVE -32], [MOVE +32], [MOVE -4], INSERT `She', INSERT `also', INSERT `taught', INSERT `with', INSERT `several', INSERT `dance', INSERT `classes', INSERT `during', INSERT `her', INSERT `time', INSERT `in', INSERT `the', INSERT `private', INSERT `private', INSERT `classes', INSERT `.', INSERT `She', INSERT `has', INSERT `taught', INSERT `classes', INSERT `both', INSERT `in', INSERT `the', INSERT `private', INSERT `and', INSERT `semi', INSERT `private', INSERT `areas', [MOVE +8], INSERT `more', INSERT `than', [MOVE -2], INSERT `have', [MOVE -8], [MOVE -16], INSERT `and', [MOVE -16], INSERT `teaching', INSERT `at', INSERT `the', INSERT `age', INSERT `of', INSERT `two', [MOVE +32], INSERT `She', INSERT `has', INSERT `taught', INSERT `for', INSERT `years', [MOVE -64], [MOVE -8], INSERT `esville', INSERT `,', INSERT `Texas', INSERT `.', INSERT `She', INSERT `taught', INSERT `at', INSERT `various', INSERT `Dance', INSERT `Studio', [MOVE -2], INSERT `studios', INSERT `including', INSERT `New', INSERT `York', INSERT `City', INSERT `Dance', INSERT `Center', INSERT `,', INSERT `the', INSERT `Brooklyn', INSERT `School', INSERT `of', INSERT `Dance', INSERT `,', INSERT `and', INSERT `the', INSERT `Center', INSERT `for', INSERT `Arts', INSERT `&', INSERT `Dance', INSERT `at', INSERT `Syracuse', [MOVE -32], STOP

100M Reviser Example 2

Text (prompt segment in blue).

Do you want to hire a trusted Professional Cleaners in Mitcham Wandsworth London SW17'
Hire our dependable Professional Cleaners company in Mitcham Wandsworth London Wandsworth for
a free no-obligation quote. With 15 years of experience, we're a professional cleaning company
and have experience with everything from carpeting, cleaning, cleaning to cleaning and so forth.
For a complete list of the services available, please contact us today.
If you'd like to hire, we'll be able to look at your work and bring you the best solution to
your requirements. We will also assist you.
We know that working with an experienced man and team will make the key difference.

Restoration trajectory (186 actions).

INSERT `Do', INSERT `you', INSERT `want', INSERT `to', INSERT `hire', INSERT `a', INSERT `trusted',
INSERT `Professional', INSERT `Clean', INSERT `ers', INSERT `in', INSERT `Mitch', INSERT `am', INSERT `W',
INSERT `ands', INSERT `worth', INSERT `London', INSERT `SW', INSERT `17', INSERT `', INSERT `\\n', INSERT
`H', INSERT `ire', INSERT `our', INSERT `depend', INSERT `able', INSERT `Professional', INSERT `Clean',
INSERT `ers', INSERT `company', INSERT `in', INSERT `Mitch', INSERT `am', INSERT `W', INSERT `ands',
INSERT `.', [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], INSERT `the', [MOVE -1], INSERT `.',
INSERT `\\n', INSERT `We', INSERT `know', [MOVE -2], [MOVE -2], INSERT `you', [MOVE +4], [MOVE +1], INSERT `key',
INSERT `difference', [MOVE -8], INSERT `', INSERT `', INSERT `ll', INSERT `be', INSERT `able',
INSERT `to', INSERT `and', INSERT `bring', INSERT `you', INSERT `your', INSERT `requirements', INSERT
`.', INSERT `We', INSERT `will', INSERT `also', INSERT `assist', [MOVE -16], [MOVE +8], [MOVE -2], INSERT
`look', INSERT `at', INSERT `work', [MOVE -1], INSERT `your', [MOVE +16], INSERT `that', INSERT `working',
INSERT `with', INSERT `an', INSERT `experienced', INSERT `man', INSERT `and', INSERT `team',
INSERT `will', INSERT `make', [MOVE -8], [MOVE -16], [MOVE +4], [MOVE +2], [MOVE -4], INSERT `the', INSERT
`best', INSERT `solution', INSERT `to', [MOVE -16], INSERT `d', INSERT `like', INSERT `to', INSERT `hire',
INSERT `,', INSERT `we', INSERT `', [MOVE -8], INSERT `a', INSERT `\\n', INSERT `If', INSERT `you',
INSERT `', [MOVE -4], INSERT `professional', INSERT `services', [MOVE -1], INSERT `the', [MOVE -2], [MOVE
-1], INSERT `worth', INSERT `London', INSERT `W', INSERT `ands', INSERT `worth', INSERT `for', INSERT `a',
INSERT `free', INSERT `no', INSERT `-', INSERT `ob', INSERT `lig', INSERT `ation', INSERT `quote', INSERT
`.', INSERT `With', INSERT `15', INSERT `years', INSERT `of', INSERT `experience', INSERT `', INSERT `we',
INSERT `', INSERT `', INSERT `re', [MOVE +2], INSERT `cleaning', INSERT `company', INSERT `with',
INSERT `everything', INSERT `from', INSERT `carpet', INSERT `ing', INSERT `,', INSERT `cleaning', INSERT
`.', INSERT `cleaning', INSERT `to', INSERT `cleaning', INSERT `and', INSERT `so', INSERT `forth',
INSERT `.', INSERT `\\n', INSERT `For', INSERT `a', INSERT `complete', INSERT `list', INSERT `of', [MOVE
+2], INSERT `available', INSERT `', INSERT `please', INSERT `contact', INSERT `us', INSERT `today',
INSERT `.', [MOVE +32], [MOVE -64], [MOVE +2], INSERT `and', INSERT `have', INSERT `experience', [MOVE -1],
[MOVE -32], STOP

100M Reviser Example 3

Text (prompt segment in blue).

5 beds | 3 baths | 2,340 sqft | \$597 per sq. ft.

Run, dont walk. Come see this beautiful custom home near the beach!

This is a beautiful home, and this home has a beautiful fireplace in the main living room and a large open living room. There is a nice large dining room, a dining room, full kitchen, a large gas stove the master bathroom with and a living. The entire home features room a gorgeous outdoor dining room. The 2 bedrooms are spacious and open-plan, and open-plan! Great location;; open concept kitchen convenient in the community. Open floor plan, walk-in closet & den. The location! This is a small town, and is a short- walk from the beach. Come to enjoy!

Restoration trajectory (192 actions).

INSERT `5', INSERT `beds', INSERT `|', INSERT `3', INSERT `baths', INSERT `|', INSERT `2', INSERT `',
INSERT `340', INSERT `sq', INSERT `ft', INSERT `|', INSERT `\$', INSERT `597', INSERT `per', INSERT `sq',
INSERT `', INSERT `ft', INSERT `', INSERT `n', INSERT `Run', INSERT `', INSERT `dont', INSERT `walk',
INSERT `', INSERT `Come', INSERT `see', INSERT `this', INSERT `beautiful', INSERT `custom', INSERT `
home', INSERT `near', INSERT `the', INSERT `beach', INSERT `!', INSERT `and', [MOVE -1], [MOVE +1], [MOVE
-1], [MOVE +1], INSERT `-', INSERT `!', [MOVE -2], INSERT `is', INSERT `a', INSERT `short', [MOVE -4], [MOVE
+4], [MOVE -4], INSERT `', [MOVE -1], INSERT `is', INSERT `a', INSERT `small', INSERT `town', [MOVE +2],
[MOVE +4], INSERT `to', INSERT `enjoy', [MOVE -2], INSERT `walk', INSERT `from', INSERT `the', INSERT `
beach', INSERT `', INSERT `Come', [MOVE -16], INSERT `!', INSERT `This', [MOVE -2], INSERT `n', INSERT
`This', INSERT `is', INSERT `a', INSERT `beautiful', INSERT `home', INSERT `', INSERT `and', INSERT `
this', INSERT `home', INSERT `has', INSERT `a', INSERT `beautiful', INSERT `fireplace', INSERT `in',
INSERT `the', INSERT `main', INSERT `living', INSERT `room', INSERT `and', INSERT `a', INSERT `large',
INSERT `open', INSERT `living', INSERT `room', INSERT `', INSERT `There', INSERT `is', INSERT `a',
INSERT `nice', INSERT `large', INSERT `dining', INSERT `room', INSERT `', INSERT `full', INSERT `
kitchen', INSERT `', INSERT `a', INSERT `large', INSERT `gas', INSERT `stove', INSERT `and', INSERT `a',
INSERT `living', INSERT `room', INSERT `a', INSERT `gorgeous', INSERT `!', INSERT `Great', INSERT `
location', [MOVE -4], [MOVE +1], INSERT `outdoor', INSERT `dining', INSERT `room', INSERT `', INSERT `
The', INSERT `2', INSERT `bedrooms', INSERT `are', INSERT `spacious', INSERT `and', INSERT `open',
INSERT `-', INSERT `plan', INSERT `', INSERT `and', INSERT `open', INSERT `-', INSERT `plan', [MOVE +2],
INSERT `location', INSERT `', INSERT `convenient', INSERT `in', INSERT `the', [MOVE -32], [MOVE +32],
INSERT `community', INSERT `', INSERT `Open', INSERT `floor', INSERT `plan', INSERT `', INSERT `walk',
INSERT `-', INSERT `in', INSERT `closet', INSERT `&', INSERT `den', INSERT `', INSERT `The', [MOVE -1],
[MOVE +16], [MOVE -32], INSERT `;', INSERT `open', INSERT `concept', INSERT `kitchen', [MOVE -32], INSERT `
the', INSERT `master', INSERT `bathroom', INSERT `with', [MOVE +1], [MOVE +2], INSERT `', INSERT `The',
INSERT `entire', INSERT `home', INSERT `features', [MOVE -16], [MOVE -2], [MOVE -2], INSERT `', INSERT `
a', INSERT `dining', INSERT `room', [MOVE -8], [MOVE +2], [MOVE -32], Stop

100M Reviser Example 4

Text (prompt segment in blue).

Being able to present the findings of research into how effective Cognitive Behavioural Therapy is in a digital and online setting - specifically for treating youth anxiety - and in comparison to a traditional mental therapy approach, it has led to new challenges for children with cognitive disorders: they are now at the forefront of psychological change; that the technology can make changes to behavior of their children, the social and behavioral problems. The authors report that brain-related research, or in the course of memory, can provide a comprehensive assessment of the effective therapy with their personal and psychological concerns, their own self, and this are a new challenge.

For more information on the full text of the authors' case studies, please contact first to share article(s) of their authors, and submitting a review by.

Restoration trajectory (228 actions).

INSERT `Being', INSERT `able', INSERT `to', INSERT `present', INSERT `the', INSERT `findings', INSERT `
of', INSERT `research', INSERT `into', INSERT `how', INSERT `effective', INSERT `Cognitive', INSERT `
Beh', INSERT `aviour', INSERT `al', INSERT `Therapy', INSERT `is', INSERT `in', INSERT `a', INSERT `
digital', INSERT `and', INSERT `online', INSERT `setting', INSERT `-', INSERT `specifically', INSERT `

for', INSERT `treating', INSERT `youth', INSERT `anxiety', INSERT `-', INSERT `and', INSERT `in', INSERT `comparison', INSERT `to', INSERT `a', INSERT `.', [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], INSERT `by', [MOVE -1], INSERT `and', [MOVE +1], [MOVE -1], [MOVE -1], INSERT `\n', [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], INSERT `For', [MOVE -2], [MOVE +2], [MOVE -2], INSERT `.', [MOVE -1], INSERT `are', [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], INSERT `with', [MOVE -1], [MOVE +2], [MOVE -2], [MOVE +4], [MOVE +1], INSERT `the', INSERT `of', [MOVE -1], INSERT `full', INSERT `.', INSERT `please', [MOVE +2], INSERT `a', INSERT `review', [MOVE -8], INSERT `more', INSERT `information', INSERT `on', [MOVE -4], [MOVE -4], INSERT `therapy', [MOVE +2], [MOVE -1], [MOVE -2], INSERT `can', [MOVE +2], [MOVE +1], INSERT `a', INSERT `new', INSERT `challenge', [MOVE +8], INSERT `text', INSERT `of', INSERT `the', INSERT `authors', INSERT `', INSERT `', INSERT `case', INSERT `studies', [MOVE +4], INSERT `submitting', [MOVE -2], INSERT `their', INSERT `authors', INSERT `.', [MOVE -4], INSERT `contact', INSERT `first', INSERT `to', INSERT `share', INSERT `article', INSERT `(', INSERT `s', INSERT `)', [MOVE -32], INSERT `provide', INSERT `the', [MOVE +2], INSERT `their', INSERT `their', INSERT `own', INSERT `self', INSERT `.', INSERT `and', INSERT `this', [MOVE -4], [MOVE -8], [MOVE +2], INSERT `a', INSERT `comprehensive', INSERT `assessment', INSERT `of', [MOVE +1], INSERT `effective', [MOVE +2], [MOVE +1], INSERT `personal', INSERT `and', INSERT `psychological', INSERT `concerns', INSERT `.', [MOVE -16], INSERT `.', INSERT `The', INSERT `authors', INSERT `report', INSERT `that', INSERT `brain', INSERT `-', INSERT `related', INSERT `research', INSERT `.', INSERT `or', INSERT `in', INSERT `the', INSERT `course', INSERT `of', INSERT `memory', INSERT `.', [MOVE -2], [MOVE -8], [MOVE +64], [MOVE -4], [MOVE -1], [MOVE -64], [MOVE -2], INSERT `and', INSERT `behavioral', INSERT `problems', [MOVE -1], [MOVE -2], INSERT `to', INSERT `social', [MOVE -2], INSERT `therapy', INSERT `approach', INSERT `.', INSERT `has', INSERT `led', [MOVE +1], INSERT `new', INSERT `challenges', INSERT `for', INSERT `children', INSERT `with', INSERT `cognitive', INSERT `disorders', INSERT `:', INSERT `the', [MOVE +1], [MOVE -2], INSERT `they', INSERT `are', INSERT `.', [MOVE -1], INSERT `now', INSERT `at', INSERT `the', INSERT `forefront', INSERT `of', INSERT `psychological', INSERT `change', INSERT `;', INSERT `that', INSERT `the', INSERT `technology', INSERT `can', INSERT `make', INSERT `changes', INSERT `to', INSERT `behavior', INSERT `of', INSERT `their', INSERT `children', [MOVE -32], INSERT `it', [MOVE -4], INSERT `traditional', INSERT `mental', [MOVE -2], STOP

100M Reviser Example 5

Text (prompt segment in blue).

This article describes the international safety standards that iPhone batteries meet. Underwriters Laboratory (UL) 2054: Covers safety of lithium-ion batteries in general use. This is the standard of safety, but it is also strictly acceptable for batteries in general use. The Standard refers to some battery manufacturers which other manufacturers must rely on the standard as a result of the standard. On other hand, if an iPhone is used to use a battery to replace or replace an old battery, there is no better the more reliable way to use the standard. As with other standard and in of batteries, safety of battery and battery batteries use in the case of battery performance issues. The standards used in this standard are described as part of the strict quality of the standard.

Restoration trajectory (202 actions).

INSERT `This', INSERT `article', INSERT `describes', INSERT `the', INSERT `international', INSERT `safety', INSERT `standards', INSERT `that', INSERT `iPhone', INSERT `batteries', INSERT `meet', INSERT `.', INSERT `\n', INSERT `Under', INSERT `writers', INSERT `Laboratory', INSERT `(', INSERT `UL', INSERT `)', INSERT `20', INSERT `54', INSERT `:', INSERT `Co', INSERT `vers', INSERT `safety', INSERT `of', INSERT `lithium', INSERT `-', INSERT `ion', INSERT `batteries', INSERT `in', INSERT `general', INSERT `use', INSERT `.', INSERT `\n', INSERT `.', [MOVE -1], [MOVE +1], [MOVE -1], INSERT `the', INSERT `standard', [MOVE -2], [MOVE +1], [MOVE -1], INSERT `the', [MOVE -1], [MOVE +1], [MOVE -1], INSERT `.', INSERT `of', INSERT `batteries', INSERT `use', [MOVE -4], [MOVE +1], [MOVE -1], INSERT `batteries', [MOVE +4], INSERT `in', [MOVE +1], INSERT `case', [MOVE -8], INSERT `of', [MOVE +2], INSERT `safety', [MOVE -4], INSERT `.', INSERT `On', INSERT `and', INSERT `in', [MOVE +4], [MOVE -8], INSERT `as', INSERT `a', INSERT `result', INSERT `of', INSERT `the', INSERT `standard', [MOVE -4], [MOVE -2], INSERT `other', INSERT `manufacturers', INSERT `must', INSERT `rely', INSERT `on', INSERT `the', INSERT `standard', [MOVE +8], INSERT `the', [MOVE -16], INSERT `.', [MOVE -1], INSERT `.', INSERT `but', INSERT `it', INSERT `is', INSERT `also', INSERT `strictly', INSERT `acceptable', INSERT `for', INSERT `batteries', INSERT `in', INSERT `general', INSERT `use', [MOVE +32], [MOVE -8], INSERT `battery', [MOVE -4], [MOVE -16], [MOVE -8], INSERT `The', INSERT `Standard', INSERT `refers', INSERT `to', INSERT `some', INSERT `battery', INSERT `manufacturers', INSERT `which', [MOVE -16], [MOVE -4], [MOVE -1], INSERT `safety', [MOVE -1], INSERT `the', INSERT `standard', INSERT `of', [MOVE -1], [MOVE -2], INSERT `This', INSERT `is', [MOVE +32], [MOVE +8], INSERT `other', INSERT `hand', INSERT `.', INSERT `if', INSERT `an', INSERT `iPhone', INSERT `is', INSERT `used', INSERT `to', INSERT `use', INSERT `a', INSERT `battery', INSERT `to', INSERT `replace',

INSERT `or', INSERT `replace', INSERT `an', INSERT `old', INSERT `battery', INSERT `.', INSERT `there',
 INSERT `is', INSERT `no', INSERT `better', [MOVE +1], INSERT `more', INSERT `reliable', INSERT `way',
 INSERT `to', INSERT `use', INSERT `the', INSERT `standard', INSERT `.', INSERT `\n', INSERT `As', INSERT `
 with', INSERT `other', INSERT `standard', [MOVE +8], INSERT `and', INSERT `battery', [MOVE +4], [MOVE +1],
 INSERT `of', INSERT `battery', INSERT `performance', INSERT `issues', INSERT `.', INSERT `The', INSERT `
 standards', INSERT `used', INSERT `in', INSERT `this', INSERT `standard', INSERT `are', INSERT `
 described', INSERT `as', INSERT `part', INSERT `of', INSERT `the', INSERT `strict', INSERT `quality',
 INSERT `of', [MOVE -2], [MOVE -64], [MOVE -2], [MOVE +8], [MOVE +8], [MOVE -64], Stop

E.2 300M Examples

300M Reviser Example 1

Text (prompt segment in blue).

Home to 279 units, Scarborough Wood Condos offers one and two bedroom + den suites. They range in size from 860 sq ft to 1120 sq ft. Building amenities include a fitness center, a fitness center, and private parking available.

Information is deemed correct at the time of publishing and is subject to change. Real estate listings obtained from third party sources are for consumers' personal purchasing decisions and should not be relied upon for any purpose other than to identify prospective properties consumers may be interested in purchasing.

The MLS should correct floor plans as required by the listing agents; however, all information provided by the listing agent may be different. Not all properties are the same and the brokers may change. Information should be independently verified, accuracy and accuracy are provided. All information is supplied by the MLS®, a program of the MLS. We do not guarantee the accuracy of this information. Please call us today!

Copyright 2016. All Rights Reserved.

Restoration trajectory (230 actions).

INSERT `Home', INSERT `to', INSERT `279', INSERT `units', INSERT `.', INSERT `Scarborough', INSERT `Wood',
 INSERT `Cond', INSERT `os', INSERT `offers', INSERT `one', INSERT `and', INSERT `two', INSERT `bedroom',
 INSERT `+', INSERT `den', INSERT `suites', INSERT `.', INSERT `They', INSERT `range', INSERT `in',
 INSERT `size', INSERT `from', INSERT `8', INSERT `60', INSERT `sq', INSERT `ft', INSERT `to', INSERT `
 1', INSERT `120', INSERT `sq', INSERT `ft', INSERT `.', INSERT `Building', INSERT `amenities', INSERT `.',
 [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], INSERT `.', INSERT `We', INSERT `do', INSERT `not',
 INSERT `guarantee', INSERT `the', INSERT `accuracy', INSERT `this', INSERT `information', INSERT `.',
 INSERT `Please', INSERT `call', INSERT `us', INSERT `today', INSERT `!', INSERT `.', INSERT `All', INSERT `
 `Rights', INSERT `Reserved', [MOVE -4], [MOVE -8], INSERT `of', [MOVE -8], INSERT `the', INSERT `MLS',
 [MOVE +16], INSERT `\n', INSERT `Copyright', INSERT `2016', [MOVE -4], [MOVE -16], [MOVE -1], [MOVE +1],
 [MOVE -1], INSERT `is', INSERT `supplied', INSERT `by', [MOVE +2], INSERT `@', INSERT `a', INSERT `MLS',
 [MOVE -8], INSERT `information', [MOVE -1], INSERT `provided', INSERT `.', INSERT `\n', INSERT `All', [MOVE
 -4], INSERT `as', INSERT `required', [MOVE -2], INSERT `and', INSERT `floor', [MOVE -2], [MOVE +2], INSERT
 `plans', [MOVE +2], INSERT `by', INSERT `the', INSERT `listing', INSERT `agents', INSERT `;', INSERT `
 however', INSERT `.', INSERT `all', INSERT `information', INSERT `provided', INSERT `by', INSERT `the',
 INSERT `listing', INSERT `agent', INSERT `and', INSERT `the', INSERT `brokers', INSERT `may', INSERT `
 change', INSERT `.', INSERT `Information', INSERT `should', INSERT `be', INSERT `independently', INSERT `
 verified', INSERT `.', INSERT `accuracy', INSERT `and', INSERT `accuracy', INSERT `are', [MOVE -16],
 INSERT `may', INSERT `be', INSERT `different', INSERT `.', INSERT `Not', INSERT `all', INSERT `
 properties', INSERT `are', INSERT `the', INSERT `same', [MOVE +32], [MOVE -4], INSERT `program', INSERT `
 of', INSERT `the', [MOVE +4], [MOVE -64], INSERT `available', INSERT `.', INSERT `\n', INSERT `Information',
 INSERT `is', INSERT `deemed', INSERT `correct', [MOVE +1], INSERT `is', INSERT `subject', INSERT `to',
 INSERT `change', INSERT `.', INSERT `Real', INSERT `consumers', INSERT `"`, INSERT `personal', INSERT `
 purchasing', INSERT `decisions', INSERT `and', INSERT `should', INSERT `not', INSERT `be', INSERT `
 relied', INSERT `upon', INSERT `for', INSERT `any', INSERT `purpose', INSERT `other', INSERT `than',
 INSERT `to', INSERT `identify', INSERT `prospective', INSERT `properties', INSERT `consumers', INSERT `
 may', INSERT `be', INSERT `interested', INSERT `in', INSERT `purchasing', INSERT `.', INSERT `\n', INSERT
 `The', INSERT `MLS', INSERT `should', INSERT `correct', [MOVE -32], INSERT `estate', INSERT `listings',
 INSERT `obtained', INSERT `from', INSERT `third', INSERT `party', INSERT `sources', INSERT `are', INSERT
 `for', [MOVE -16], INSERT `at', INSERT `the', INSERT `time', INSERT `of', INSERT `publishing', [MOVE +4],
 [MOVE -16], INSERT `include', INSERT `a', INSERT `parking', [MOVE -1], INSERT `fitness', INSERT `center',
 INSERT `.', INSERT `a', INSERT `fitness', INSERT `center', INSERT `.', INSERT `and', INSERT `private',

[MOVE -4], [MOVE +4], [MOVE -8], [MOVE -2], [MOVE -1], Stop

300M Reviser Example 2

Text (prompt segment in blue).

with the new year, new content comes along. The Argent Dawn will start its operations in the Plaguelands on the 23rd of January - unlocking new missions to complete.

We will be adding more games and activities in the coming months, so make sure that you follow our page on Facebook and Twitter in order to get more information on the mission. We will be happy to help you with the new content.

As well as these new missions, we look forward to continue working on this new mission for more people!

This is a great time for our customers. Thanks to everyone who has supported us.

As always, we are happy to use this opportunity to thank all those who have provided feedback, input and suggestions. Thank you.

Restoration trajectory (203 actions).

INSERT `with', INSERT `the', INSERT `new', INSERT `year', INSERT `,', INSERT `new', INSERT `content',
INSERT `comes', INSERT `along', INSERT `.', INSERT `The', INSERT `Argent', INSERT `Dawn', INSERT `will',
INSERT `start', INSERT `its', INSERT `operations', INSERT `in', INSERT `the', INSERT `Pl', INSERT `ag',
INSERT `uel', INSERT `ands', INSERT `on', INSERT `the', INSERT `23', INSERT `rd', INSERT `of', INSERT `January',
INSERT `- ', INSERT `unlocking', INSERT `new', INSERT `missions', INSERT `to', INSERT `complete',
INSERT `.', [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1],
[MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], [MOVE -1], INSERT `you', [MOVE -1], [MOVE
+2], [MOVE -2], INSERT `Thank', [MOVE -1], INSERT `to', INSERT `use', INSERT `.', [MOVE -1], INSERT `this',
INSERT `opportunity', INSERT `to', INSERT `thank', INSERT `all', INSERT `those', INSERT `who', INSERT `have',
INSERT `provided', INSERT `feedback', INSERT `,', INSERT `input', INSERT `and', INSERT `suggestions',
[MOVE -16], INSERT `our', [MOVE -1], INSERT `for', [MOVE +1], INSERT `customers', INSERT `happy',
[MOVE -4], INSERT `time', [MOVE -1], INSERT `this', INSERT `new', INSERT `mission', INSERT `for',
INSERT `more', [MOVE +8], [MOVE -8], INSERT `people', INSERT `!', INSERT `\n', INSERT `is', INSERT `a',
INSERT `great', [MOVE +4], INSERT `,', INSERT `we', INSERT `are', [MOVE +16], [MOVE -32], [MOVE -2], [MOVE
+1], [MOVE -1], [MOVE +2], [MOVE -2], INSERT `to', INSERT `continue', [MOVE -2], INSERT `with', INSERT `the',
INSERT `new', INSERT `content', INSERT `.', INSERT `\n', INSERT `As', INSERT `well', INSERT `as',
INSERT `these', INSERT `new', INSERT `missions', INSERT `,', INSERT `we', INSERT `look', INSERT `forward',
[MOVE -16], INSERT `the', INSERT `mission', INSERT `.', INSERT `be', INSERT `happy', INSERT `to',
INSERT `help', INSERT `you', [MOVE -8], INSERT `on', [MOVE -1], INSERT `more', INSERT `information',
[MOVE +4], INSERT `We', INSERT `will', [MOVE -8], INSERT `.', INSERT `\n', INSERT `We', INSERT `will',
INSERT `be', INSERT `adding', INSERT `more', INSERT `games', INSERT `and', INSERT `activities', INSERT `in',
INSERT `the', INSERT `coming', INSERT `months', INSERT `get', [MOVE +32], [MOVE -1], INSERT `working',
INSERT `on', [MOVE +8], INSERT `This', [MOVE -1], [MOVE +8], INSERT `.', INSERT `Thanks', INSERT `to',
INSERT `everyone', INSERT `who', INSERT `has', INSERT `supported', INSERT `us', INSERT `.', INSERT
`\n', INSERT `As', INSERT `always', [MOVE -32], [MOVE -32], [MOVE +2], INSERT `,', INSERT `so', INSERT `make',
INSERT `sure', INSERT `that', INSERT `you', INSERT `follow', INSERT `our', INSERT `page', INSERT
`on', INSERT `Facebook', INSERT `and', INSERT `Twitter', INSERT `in', INSERT `order', INSERT `to',
[MOVE +2], [MOVE -32], Stop

300M Reviser Example 3

Text (prompt segment in blue).

Go from black screen from your security cameras to full coverage of your retail or commercial security system with our security camera repair service.

Our customers are at the center of our universe and our service is second to none. Security cameras come to us when their needs aren't met. We can also repair or replace entire systems or use additional security equipment. We are passionate about the safety and security of our customers, and we are proud of our reputation. If you need an solution to your security camera immediate repair problem, please contact us.

Our goal is to give you a professional, cost-effective and affordable solution that is suitable for you. Our professional technicians have the time and expertise to provide you with excellent

solutions, and the best prices and solutions available. To learn more, visit our website to learn more about our company and your security solutions.

Restoration trajectory (200 actions).

```
INSERT `Go', INSERT `from', INSERT `black', INSERT `screen', INSERT `from', INSERT `your', INSERT `security', INSERT `cameras', INSERT `to', INSERT `full', INSERT `coverage', INSERT `of', INSERT `your', INSERT `retail', INSERT `or', INSERT `commercial', INSERT `security', INSERT `system', INSERT `with', INSERT `our', INSERT `security', INSERT `camera', INSERT `repair', INSERT `service', INSERT `.', INSERT `n', INSERT `Our', INSERT `customers', INSERT `are', INSERT `at', INSERT `the', INSERT `center', INSERT `of', INSERT `our', INSERT `universe', INSERT `.', [MOVE -1], [MOVE +1], [MOVE -1], INSERT `you', [MOVE +1], [MOVE -2], INSERT `n', [MOVE -1], INSERT `and', [MOVE +1], INSERT `Our', INSERT `goal', INSERT `is', INSERT `to', INSERT `give', [MOVE +1], INSERT `a', INSERT `professional', INSERT `.', INSERT `cost', INSERT `solutions', [MOVE -8], [MOVE -4], INSERT `.', [MOVE -2], INSERT `.', [MOVE -1], INSERT `our', INSERT `customers', [MOVE +2], INSERT `we', INSERT `are', INSERT `proud', INSERT `of', INSERT `our', INSERT `reputation', INSERT `.', INSERT `If', INSERT `you', INSERT `need', INSERT `an', INSERT `immediate', INSERT `repair', INSERT `.', INSERT `please', INSERT `us', [MOVE -1], INSERT `contact', [MOVE +1], [MOVE -4], INSERT `problem', [MOVE -16], [MOVE -2], INSERT `and', INSERT `our', INSERT `service', INSERT `is', INSERT `second', INSERT `to', INSERT `none', INSERT `.', INSERT `Security', INSERT `cameras', INSERT `come', INSERT `to', INSERT `us', INSERT `when', INSERT `their', INSERT `needs', INSERT `aren', INSERT `t', INSERT `of', [MOVE -1], INSERT `met', INSERT `.', INSERT `We', INSERT `can', INSERT `also', INSERT `repair', INSERT `or', INSERT `replace', INSERT `entire', INSERT `systems', INSERT `or', INSERT `equipment', INSERT `.', INSERT `We', INSERT `are', INSERT `passionate', INSERT `about', INSERT `the', INSERT `safety', INSERT `and', INSERT `security', [MOVE +32], [MOVE +4], [MOVE -1], INSERT `-', INSERT `effective', INSERT `and', INSERT `affordable', INSERT `security', [MOVE -1], INSERT `solution', INSERT `that', INSERT `is', INSERT `suitable', INSERT `for', INSERT `you', INSERT `.', INSERT `Our', INSERT `professional', INSERT `technicians', INSERT `have', INSERT `your', [MOVE -1], INSERT `the', INSERT `time', INSERT `and', INSERT `expertise', INSERT `to', INSERT `provide', INSERT `you', INSERT `with', INSERT `excellent', INSERT `solutions', INSERT `.', INSERT `and', INSERT `the', INSERT `best', INSERT `prices', INSERT `and', INSERT `solutions', INSERT `available', INSERT `.', INSERT `To', INSERT `learn', INSERT `more', INSERT `.', INSERT `visit', INSERT `our', INSERT `our', INSERT `company', INSERT `and', [MOVE +1], [MOVE -4], INSERT `website', INSERT `to', INSERT `learn', INSERT `more', INSERT `about', [MOVE -64], INSERT `solution', INSERT `to', INSERT `your', INSERT `security', INSERT `camera', [MOVE +1], [MOVE -32], INSERT `use', INSERT `additional', INSERT `security', [MOVE -32], Stop
```

300M Reviser Example 4

Text (prompt segment in blue).

Mobile optimization is one of the top priorities for any website now-a-days because of increased traffic & conversions from devices like mobiles & tablets. So, folks who wish to get their business on Google and Google+ can also boost the ranking of your site in the search engines which will increase the visibility & engagement of your website and result more traffic and revenue to your website. And, it will also get search engines to believe in their website. Whether you are looking for a complete mobile optimization for your website or simply want to know more about our mobile optimisation services now, we will provide you with the best mobile internet and mobile optimization services in your area.

Our website optimization services are designed to ensure that your page is accessible in all browsers and if your website is not working efficiently, we will work with you to improve your page speed.

Restoration trajectory (202 actions).

```
INSERT `Mobile', INSERT `optimization', INSERT `is', INSERT `one', INSERT `of', INSERT `the', INSERT `top', INSERT `priorities', INSERT `for', INSERT `any', INSERT `website', INSERT `now', INSERT `-', INSERT `a', INSERT `-', INSERT `days', INSERT `because', INSERT `of', INSERT `increased', INSERT `traffic', INSERT `&', INSERT `conversions', INSERT `from', INSERT `devices', INSERT `like', INSERT `mob', INSERT `iles', INSERT `&', INSERT `tablets', INSERT `.', INSERT `So', INSERT `.', INSERT `folks', INSERT `who', INSERT `wish', INSERT `optim', [MOVE -1], [MOVE +1], [MOVE -1], [MOVE +1], INSERT `you', INSERT `.', [MOVE -1], [MOVE -2], INSERT `our', INSERT `mobile', [MOVE +1], [MOVE +2], [MOVE -2], INSERT `isation', INSERT `services', INSERT `now', INSERT `.', INSERT `with', [MOVE -1], INSERT `we', INSERT `will', INSERT `provide', INSERT `you', INSERT `with', INSERT `the', INSERT `best', INSERT `mobile', INSERT `internet', [MOVE -16], INSERT `website', INSERT `a', INSERT `complete', INSERT `website', INSERT `or', INSERT `simply', INSERT `want', INSERT `to', INSERT `know', INSERT `more', INSERT `about', [MOVE +16], INSERT `and', INSERT `mobile', INSERT `optimization', INSERT `services', INSERT `in', INSERT `your', INSERT `
```

area', INSERT `.', INSERT `\\n', INSERT `Our', INSERT `website', INSERT `optimization', INSERT `services', INSERT `are', INSERT `designed', INSERT `to', INSERT `ensure', INSERT `that', INSERT `your', INSERT `page', INSERT `is', INSERT `.', INSERT `we', INSERT `will', INSERT `work', [MOVE +2], INSERT `to', INSERT `improve', [MOVE -8], INSERT `accessible', INSERT `in', INSERT `all', INSERT `browsers', INSERT `and', INSERT `if', INSERT `your', INSERT `website', INSERT `is', INSERT `not', INSERT `working', INSERT `efficiently', [MOVE +8], INSERT `your', INSERT `page', INSERT `speed', [MOVE -64], [MOVE -4], INSERT `for', INSERT `your', [MOVE -4], INSERT `.', INSERT `\\n', INSERT `Whether', INSERT `you', INSERT `are', INSERT `looking', INSERT `for', [MOVE -8], INSERT `to', INSERT `get', INSERT `their', INSERT `business', INSERT `on', INSERT `Google', INSERT `and', INSERT `their', [MOVE -1], INSERT `Google', INSERT `+', INSERT `can', INSERT `also', INSERT `boost', INSERT `the', INSERT `ranking', INSERT `of', INSERT `in', [MOVE +1], [MOVE -2], INSERT `your', INSERT `site', INSERT `in', INSERT `the', INSERT `search', INSERT `engines', INSERT `which', INSERT `will', INSERT `increase', INSERT `the', INSERT `visibility', INSERT `&', INSERT `engagement', INSERT `of', INSERT `your', INSERT `website', INSERT `and', INSERT `result', INSERT `more', INSERT `traffic', INSERT `and', INSERT `revenue', INSERT `to', INSERT `your', INSERT `website', INSERT `.', INSERT `And', INSERT `also', INSERT `get', INSERT `search', INSERT `engines', INSERT `to', INSERT `believe', [MOVE -2], [MOVE -4], INSERT `.', INSERT `it', INSERT `will', [MOVE -16], [MOVE +32], [MOVE +1], [MOVE +1], INSERT `mobile', INSERT `optimization', [MOVE -64], [MOVE -1], Stop

300M Reviser Example 5

Text (prompt segment in blue).

Workplace Injury Lawyer | Robert P. Schuster, P.C.

A fundamental and appropriate expectation of any employer is that a safe workplace is provided for all employees in the workplace. This is essential, and we are available to assist you with your legal needs. We represent all types of clients in numerous criminal cases, including local, state, and federal.

Robert P. Schuster and his staff are business injury lawyers, personal injury legal, and other types of business injury and can help you in a number of different fields in every aspect of your legal career.

Contact Robert P. Schuster, P.C. at (888) 876-3088 right away.

© 2014 Robert C. Law Firm, Inc.

Restoration trajectory (171 actions).

INSERT `Work', INSERT `place', INSERT `Injury', INSERT `Law', INSERT `yer', INSERT `|', INSERT `Robert', INSERT `P', INSERT `.', INSERT `Sch', INSERT `uster', INSERT `.', INSERT `P', INSERT `.', INSERT `C', INSERT `.', INSERT `\\n', INSERT `A', INSERT `fundamental', INSERT `and', INSERT `appropriate', INSERT `expectation', INSERT `of', INSERT `any', INSERT `employer', INSERT `is', INSERT `that', INSERT `a', INSERT `safe', INSERT `workplace', INSERT `is', INSERT `provided', INSERT `for', INSERT `all', INSERT `employees', INSERT `.', [MOVE -1], [MOVE +1], INSERT `.', [MOVE -1], [MOVE -1], INSERT `.', INSERT `personal', INSERT `injury', INSERT `legal', [MOVE +1], INSERT `Inc', [MOVE -2], INSERT `.', INSERT `and', INSERT `and', INSERT `can', [MOVE -8], INSERT `injury', INSERT `lawyers', [MOVE -2], INSERT `business', [MOVE +8], INSERT `other', INSERT `types', INSERT `of', INSERT `business', INSERT `injury', [MOVE +2], INSERT `help', INSERT `you', INSERT `in', INSERT `a', INSERT `number', INSERT `of', INSERT `different', INSERT `fields', INSERT `in', INSERT `every', INSERT `aspect', INSERT `of', INSERT `your', INSERT `legal', INSERT `career', INSERT `.', INSERT `Robert', INSERT `uster', INSERT `.', INSERT `P', INSERT `.', INSERT `C', INSERT `.', INSERT `Law', INSERT `Firm', [MOVE -8], INSERT `P', INSERT `.', INSERT `Sch', [MOVE -4], INSERT `\\n', INSERT `Contact', [MOVE +8], INSERT `C', INSERT `.', INSERT `at', INSERT `(', INSERT `888', INSERT `)', INSERT `8', INSERT `76', INSERT `-', INSERT `30', INSERT `88', INSERT `right', INSERT `away', INSERT `.', INSERT `\\n', INSERT `©', INSERT `2014', INSERT `Robert', [MOVE +4], [MOVE -64], INSERT `are', INSERT `local', INSERT `.', INSERT `state', INSERT `.', INSERT `and', INSERT `federal', INSERT `.', INSERT `\\n', INSERT `Robert', INSERT `P', INSERT `.', INSERT `Sch', INSERT `uster', INSERT `and', INSERT `his', INSERT `staff', INSERT `are', [MOVE -16], [MOVE -2], INSERT `in', INSERT `the', INSERT `workplace', INSERT `.', INSERT `This', INSERT `is', INSERT `essential', INSERT `.', INSERT `and', INSERT `we', [MOVE +1], INSERT `available', INSERT `to', INSERT `assist', INSERT `you', INSERT `with', INSERT `your', INSERT `legal', INSERT `needs', INSERT `.', INSERT `We', INSERT `represent', INSERT `all', INSERT `types', INSERT `of', INSERT `clients', INSERT `in', INSERT `numerous', INSERT `criminal', INSERT `cases', INSERT `.', INSERT `including', [MOVE -32], Stop

F Reproducibility Checklist

This appendix records the key artifacts and settings needed to reproduce the reported quality, trajectory, and FLOPs results. The paper source is self-contained in this single main.tex file (no external appendix \input files are required).

F.1 Artifacts, data, and runtime

- Code repository: <https://github.com/Sean-Diab/Reviser>.
- Released Reviser checkpoints: <https://huggingface.co/sean-diab/reviser-checkpoints>.
- Environment: scripts/requirements.txt (Python package list for training, inference, evaluation, and HTML visualization scripts).
- Core runtime: PyTorch 2.11, CUDA 13, RTX 5090.
- Evaluation dataset: allenai/c4 (English validation split), GPT-2 tokenizer (use_fast=False), keep total token length in 144.00 to 216.00, fixed 35-token prompt, target total length 180.
- Evaluation set size: 1000 prompts per seed.
- Data release policy: code is released end-to-end, but raw training/evaluation data and internal checkpoints are not bundled; all scripts accept user-provided local file paths.

F.2 End-to-end code path

- Training scripts: scripts/train/train_ar.py and scripts/train/train_reviser.py.
- Inference scripts: scripts/inference/run_ar_inference.py and scripts/inference/run_reviser_inference.py.
- Evaluation scripts: scripts/eval/evalppl.py, scripts/eval/arena_from_judgments.py, and scripts/eval/trajectory_stats.py.
- HTML restoration-trajectory viewer: scripts/viz/build_restoration_trajectory_html.py.
- Single-command orchestration: scripts/reproduce_paper.py.
- Usage examples and CLI commands are documented in scripts/README.md.

F.3 Model and training

- Reviser architecture: 100M uses $(L, d, h, d_{ff}) = (24, 512, 8, 2048)$; 300M uses $(26, 896, 14, 3584)$; RoPE and tied embeddings.
- Context limits: 100M $(T_{\max}, L_{\max}) = (255, 512)$; 300M $(511, 512)$.
- Action space: $\mathcal{M} = \{\pm 1, \pm 2, \pm 4, \pm 8, \pm 16, \pm 32, \pm 64, \pm 128, \pm 256, \pm 512\}$.
- Optimizer/training: AdamW ($\beta_1 = 0.9, \beta_2 = 0.95, \epsilon = 10^{-8}, \text{wd}=0.1$), cosine schedule with 2000-step warmup, bf16, dropout 0.1, grad clip 1.0.
- Learning rates: 100M peak/min $10^{-4}/2 \times 10^{-5}$; 300M peak/min $10^{-5}/10^{-6}$.
- Effective batching: 100M (bs 64, accum 1, seq 256); 300M (bs 16, accum 4, seq 512).
- Training budget: 30B processed tokens.
- Trajectory construction: obfuscation mix $p(\text{DELETE}) = 0.8, p(\text{MOVE}) = 0.2$.

F.4 Hardware and FLOPs accounting

- Hardware: single RTX 5090 (32GB VRAM), single-node PCIe setup (no DDP/FSDP/pipeline parallelism).
- Software: PyTorch 2.11 + CUDA 13; PyTorch SDPA attention kernels; no external flash_attn.
- FLOPs protocol for Appendix metrics: dominant-matmul proxy in Appendix B with shared constants $n = 128$, $|V_c| = 50000$, $d = 768$, $L = 12$, $d_{ff} = 4d$, $K_{\max} = 16$.

F.5 Decoding and inference configuration

- Action selection: multinomial sampling with temperature $\tau = 0.9$, top- $k = 50$, and no top- p truncation.
- Stop criteria: stop on END_OF_RESPONSE (id 50260); decoding caps generation at 256 actions.
- Validity masking: enforce cursor bounds, canvas length bounds, and operator validity (e.g., DELETE invalid at cursor position 0; reserved tokens never trained/selected).
- Prompting/conditioning (if any): no special delimiter tokens; boundary is positional. Prompt prefix occupies positions $[0, 35)$ and continuation/editable region is $[35, 180)$. For SEDD/MDLM, prefix projection clamps $x[:, 0:35] = \text{prefix_ids}$ at every diffusion step; for Reviser, the prompt is provided as the first 35 inserted prompt tokens and continuation actions are generated thereafter.
- Diffusion baselines: SEDD and MDLM are decoded with 128 diffusion steps.

F.6 Evaluation protocol and reporting

- Primary quality metrics: evalPPL and with-input arena win rates; implementation/library: GPT-2 Large scoring + Skywork-Critic-8B arena judge.
- Aggregation: mean $\pm$ std over 3 seeds (123, 124, 125).